



UNIVERSIDAD
DE GRANADA

Desarrollo de Sistemas Distribuidos

Apuntes de Clase

Autor:

Tu Nombre Completo

Grado:

ADE / Ingeniería

2 de junio de 2026

Índice general

Capítulo 1

1.1. Características y objetivos

1.1.1. Compartición de recursos

- Recurso: Estos pueden ser tanto hardware como software.
- Elasticidad: Además cabe la posibilidad de una gestión elástica ,es decir, se pueden asignar mas recursos de forma momentánea.
- Políticas y métodos de gestión de recursos: Denominación ,Asociación y coordinación.
- Control descentralizado: Soporte a grupos de usuarios que trabajan cooperativamente, a esto se les llama groupware

1.1.2. Sistema abierto

- Determina si el sistema puede ser personaliza y ampliado
- Tanto software como hardware
- Unix es un ejemplo de sistema abierto
 - Desarrolladores de aplicaciones pueden usar el sistema operativo sin necesidad de pagar por ello
 - Vendedores de hardware, ya que el sistema puede ser ampliado
 - Vendedores de software y usuarios, ya que es independiente del hardware
- Utilización de protocolos estándares de comunicación, tales como IPC.

1.1.3. Concurrency

- Concurrency and parallelism arise in distributed systems
 - Activity independent of each user
 - Independence between hardware resources
 - Localization of processes in different computers

1.1.4. Scalability

- Software must be independent of size.
- The process of requesting resources must be independent of the size of the network
- Challenge, design software that is scalable, efficient and effective. Techniques to achieve it:
 - Use of caches
 - Replication of data
 - Deployment of services
 - Open systems

1.1.5. Fault tolerance

- The design is based on two proposals:
 - Hardware redundancy: If one component fails, another can take its place.
 - Software redundancy: If one process fails, another can take its place.

1.1.6. Transparency

- View the system as a whole
- Primary objective of design of distributed systems software
- Types of basic transparency:
 - Access transparency: The user does not have to worry about the location of the resources.
 - Location transparency: The user does not have to worry about the physical location of the resources.
 - Concurrency transparency: It is possible to perform concurrent operations on the same resources without interfering with each other.
- Types of advanced transparency:

- Transparencia de replicación: El usuario no debe preocuparse por la existencia de múltiples copias de un recurso.
- Transparencia de fallo: Ocultación de fallos de hardware o software.
- Transparencia de migración: Migración de entidades sin afectar a programas o usuarios.
- Transparencia de rendimiento: El sistema puede adaptarse a las cargas de trabajo variables
- Transparencia de escalabilidad: El sistema puede crecer sin cambiar las estructuras de los sistemas.

1.2. Paradigmas de Aplicaciones Distribuidas

- Aplicaciones paralelas de alto rendimiento:
 - Decrementan el tiempo de respuesta
 - Ventaja de escalabilidad frente a multiprocesadores
 - Se clasifica en cuanto al grano de paralelismo:
 - Grano fino: ++Tiempo Comunicación, –Tiempo computación.
 - Grano medio , adecuado para S.D y débilmente acoplados.
 - Grano grueso: ++Tiempo computación, –Tiempo comunicación.
 - Aplicaciones tolerancia a fallos:
 - Incrementan fiabilidad y disponibilidad
 - Propiedad de fallo parcial
 - Seguridad mediante la replicación en varios nodos
 - Aplicaciones con especialización funcional:
 - Cada aplicación es una colección de servicios
 - Forma natural de diseñar estas aplicaciones es mediante un S.D con implementaciones alternativas (replicada,distribuida,centralizada)
 - Proporcionan alto rendimiento y seguridad
 - Comunicación entre servicios
 - Fácil escalabilidad de las aplicaciones
 - Aplicaciones inherentemente distribuidas:
 - Groupware: Sistema basado en computadores que soportan la colaboración entre usuarios.
 - ◇ Ejemplos:
 - ◇ Correo electrónico
 - ◇ Software de gestión de flujos de trabajo
 - ◇ Sistema Enterprise Resource Planning (ERP)
 - ◇ Business Process Management (BPMS)

1.3. Características y objetivos emergentes en nuevos paradigmas de Computación Distribuida:

- Modularidad
- Cross platform(Interoperabilidad)
- Seguridad/privacidad
- Calidad de servicio
- Extensibilidad
- Movilidad
- Amigabilidad de las interfaces
- Elasticidad y escalabilidad
- Dispositivos móviles + LAN inalámbricas, sistemas distribuidos con clientes y tal vez servidores, móviles
- Permanecen los principios de diseño de sistemas distribuidos
- Aspectos adicionales a considerar:
 - Variaciones impredecibles en la calidad de conexión de red
 - Fiabilidad y robustez reducida de los dispositivos móviles
 - Potencias de los recursos reducida, derivada de las limitaciones de tamaño y peso
 - Consumo de batería
- Campo de investigación y desarrollo muy activo
- Áreas de trabajo:
 - Redes móviles: IP, protocolos, TCP,UDP, etc.
 - Acceso móvil a la información: Trabajo en modo desconexión,adaptación al ancho de banda, gestión de memoria.
 - Soporte a aplicaciones adaptativas: Gestión de recursos adaptativa, trans-coding.
 - Técnicas de ahorro de energía: Gestión de energía adaptativa.
 - Sensibilidad al contexto: Detección de la ubicación para determinar cambio en el comportamiento.
- Invisibilidad

- Ideal expresado por Mark Weiser
 - Minimizar las distracciones del usuario por el uso de la tecnología
 - Si la tecnología no distrae, puede usarse a un nivel subconsciente
 - Equilibrio frente a la ausencia total de notificaciones para evitar sorpresas posteriores
 - Ejemplo: Conmutación transparente entre el uso de una red Wi-Fi y una conexión 4G.
- Escalabilidad localizada
 - Al crecer el nivel de sofisticación de los espacios, aumentan las interacciones del usuario con el entorno.
 - Consecuencias en el consumo de ancho de banda, energía y aparición de distractores.
 - Escalabilidad
 - Hasta ahora no importaba la distancia o el lugar del usuario.
 - Ahora el número de interacciones debe reducirse en función de como se desplazan los usuarios.
- Enmascaramiento de la condición desigual

Comunicación y Sincronización de Procesos

2.1. Paso de Mensajes (Message Passing)

2.1.1. Conceptos y Formación de Mensajes

- **Marshalling (Formar):** Proceso de preparar datos para su transmisión.
 - **To flat (Poner en plano):** Conversión de estructuras (punteros, grafos) en secuencias lineales.
 - **Representación Estándar:**
 - **XDR (SUN):** Define tipos básicos para evitar problemas de *endianness*.
 - **CDR (CORBA):** (Aporte del libro) Los datos se envían en el formato del emisor y el receptor traduce si es necesario (“el receptor paga”).
- **Canal:** Abstracción de red física. Proporciona `send` y `receive`.

2.1.2. Sincronización y Notaciones

- **Síncrona:** `send` y `receive` bloqueantes.
- **Asíncrona:** Normalmente solo `receive` bloquea.
 - *Buffer finito:* `send` bloquea si el buffer está lleno.
 - *Buffer infinito:* `send` nunca bloquea.
- **No bloqueante:** Su ejecución nunca retrasa al proceso. El `receive` no bloqueante es complejo y requiere **sondeos (polling)** o **interrupciones**.
- **Primitivas adicionales:** `empty <puerto>` para verificar el estado de la cola.

2.2. Destinos de Comunicación y Grupos

2.2.1. Tipos de Destinos (Clasificación UGR)

- a) **Proceso:** Referencia directa, punto a punto.
- b) **Enlace (Link):** Punto a punto con indirección.
- c) **Puerto (Port):** Muchos a uno con indirección.
- d) **Buzón (Mailbox):** Muchos a muchos con indirección.
- e) **Difusión (Broadcast):** Muchos a muchos (todos).
- f) **Selección (Multicast):** Muchos a muchos (grupo).

2.2.2. Protocolos de Grupo

- **Usos:** Tolerancia a fallos, localización de objetos (ej. ficheros), rendimiento y actualización de eventos.
- **Propiedades:** **Atomicidad** (todos o ninguno) y **Ordenación** (mismo orden para todos).
- **Fiabilidad:** Construida sobre canales no fiables mediante ACKs (ej. TCP/IP).

2.3. RPC (Llamada a Procedimiento Remoto)

2.3.1. Estructura y Módulos

- **Interfaz e IDL:** Se compila la especificación IDL para generar **Stubs**.
 - *Cliente:* Convierte llamada local a remota (Marshalling).
 - *Servidor:* Selecciona y llama al procedimiento (Unmarshalling).
- **Servicio de Ligadura (Binding):** Asociación de nombre a identificador de comunicación.
 - **Exportar/Importar:** Los servidores registran, los clientes buscan.
 - **Localización del Binder:** Mediante dirección conocida, variables de entorno del SO o **Broadcast**.

2.3.2. RPC Asíncrona y Optimizaciones

- El cliente envía muchas peticiones sin esperar respuesta inmediata.
- **Ventaja:** Paralelismo y planificación eficiente en el servidor.
- **Optimización:** Agrupar varias peticiones en una comunicación (por tiempo o por necesidad de respuesta).

2.4. Protocolo Petición-Respuesta (Request-Reply)

2.4.1. Alternativas de Implementación (Coulouris)

El libro detalla tres protocolos fundamentales según el nivel de confirmación:

- **R (Request):** El cliente no espera respuesta.
- **RR (Request-Reply):** La respuesta sirve como ACK.
- **RRA (Request-Reply-Acknowledge):** El cliente confirma que recibió la respuesta. Permite al servidor descartar el histórico.

2.4.2. Modelo de Fallos y Garantías

Tabla 2.1: Garantías de envío y semánticas de invocación (Modelo de fallos)

Semántica de Invocación	Reintentar petición	Filtrar duplicados	Reejecutar o Retransmitir
Quizás (Maybe)	No	No aplicable	No aplicable
Al menos una vez (At-least-once)	Sí	No	Reejecutar procedimiento (Idempotencia, Sun RPC)
Exactamente una vez (At-most-once)	Sí	Sí	Retransmitir respuesta (RPC, RMI, CORBA)

2.5. Citas (Rendez-vous) y No Determinismo

2.5.1. No Determinismo e Instrucciones Guardadas

Uso de guardas B ; C ->S.

- **Semántica:** Éxito si B es cierto y C no retrasa; Bloqueo si B es cierto pero C debe esperar.
- **Construcciones:** **IF** (Alternativa no determinista) y **DO** (Repetitiva).

2.5.2. Citas Extendidas

Diferencia con RPC: El servidor es un ****proceso activo****.

- `call <proceso>.<operacion>` VS `in <operacion>->S`.
- El servidor puede definir distintas guardas para una misma operación.

2.5.3. Ejemplo Ada (Buffer Acotado)

```
task body Buffer1 is
begin
  loop
    select
      when (nElementos < tamBuffer) =>
        accept Escribir (Elem: TElemento) do
          ElemLocal := Elem;
          nElementos := nElementos + 1;
        end Escribir;
      or when (nElementos > 0) =>
        accept Leer (Elem: out TElemento) do
          Elem := ElemLocal;
          nElementos := nElementos - 1;
        end Leer;
      or terminate;
    end select;
  end loop;
end Buffer1;
```

Ejercicios de Sistemas Distribuidos

3.1. Asignador de Recursos

Diseño de un asignador de recursos con gestión de cola de espera manual (Paso de Mensajes).

```
1 // Definicion de canales
2 type clase_op = enum (ADQUIRIR, LIBERAR);
3 port peticion (int id_cliente, clase_op operacion, int unidad);
4 chan respuesta [1..n](int unidad_asignada);
5
6 Servidor {
7     var unidades: set of int;
8     var pendientes: queue of int;
9     var disponibles: int := MAX_UNIDADES;
10
11     while true {
12         // 1. PRIORIDAD: Si hay alguien esperando y hay recursos,
13         //    desbloquear
14         if (not empty(pendientes) && disponibles > 0) {
15             cliente = dequeue(pendientes);
16             unidad = extraer(unidades);
17             disponibles--;
18             send respuesta[cliente](unidad);
19         }
20
21         // 2. ESCUCHA: El servidor atiende peticiones
22         receive peticion(i, operacion, uni)
23         switch operacion {
24             case ADQUIRIR:
25                 if (disponibles > 0) {
26                     send respuesta[i](extraer(unidades));
27                     disponibles--;
28                 } else {
```

```

28         // Bloqueo logico: No hay send, el cliente
           espera
29         enqueue(pendientes, i);
30     }
31     case LIBERAR:
32         if (empty(pendientes)) {
33             insertar(unidades, uni);
34             disponibles++;
35         } else {
36             // Entrega directa al siguiente en la cola
37             proximo = dequeue(pendientes);
38             send respuesta[proximo](uni);
39         }
40     }
41 }
42 }

```

Listing 3.1: Asignador de Recursos

3.2. Servidor de Ficheros

Gestión de sesiones mediante canales dinámicos y mailbox.

```

1 // Canales: Mailbox para apertura, array para operaciones
2 mailbox abrir(string nombref, int id_cli);
3 chan leer[1..n](...), escribir[1..n](...), cerrar[1..n](...);
4 chan respuesta_abrir[1..n](int descriptor);
5 do receive abrir(nombref, id_cli) ->
6     // Se asigna el canal 'i' (descriptor de fichero)
7     send respuesta_abrir[id_cli](i);
8
9     do // Bucle de sesion privada para el fichero i
10         receive leer[i](datos) ->
11             read(i, datos);
12             send respuesta_cli[id_cli](datos);
13
14         [] receive escribir[i](datos) ->
15             write(i, datos);
16             send respuesta_cli[id_cli](ok);
17
18         [] receive cerrar[i]() ->
19             close(i);
20             send respuesta_cli[id_cli](bye);
21             break; // Salida del bucle interno
22     od

```

```

23     fichero_abierto[i] = false;
24 od

```

Listing 3.2: Servidor de Ficheros

3.3. Buffer Acotado

Implementación mediante Órdenes Guardadas (Estilo Ada/CSP).

```

1 Bufferacotado::
2   op poner(dato: T);
3   op tomar(var res: T);
4
5   var buffer[1..n]: T;
6   var cab: int := 1; cola: int := 1; contador: int := 0;
7
8   do true ->
9       in poner(dato) and (contador < n) ->
10          buffer[cola] := dato;
11          cola := (cola mod n) + 1;
12          contador++;
13       [] tomar(res) and (contador > 0) ->
14          res := buffer[cab];
15          cab := (cab mod n) + 1;
16          contador--;
17       nin
18   od

```

Listing 3.3: Buffer Acotado con Guardas

3.4. Ejercicio Examen Banco

Servidor con estado, tokens de seguridad y lógica de control mediante IF.

mailbox identificar(string nombre, string contra, int j); // Operaciones privadas: Un array de canales, uno para cada servidor [1..n] chan SeleccionarCuenta[1..n](int token, TCuenta cuenta, int j); chan ConsultarSaldo[1..n](int token, int j); chan IngresarDinero[1..n](int token, float cantidad, int j); chan RetirarDinero[1..n](int token, float cantidad, int j); chan BloquearCuenta[1..n](int token, int j); chan SalirCliente[1..n](int token, int j);
 // Respuestas al cliente: Un array de canales para los 'm' clientes del sistema chan respuesta_d[1..m](int token, TListalista); chan respuesta_{op}[1..m](any resultado);

```

while (true) // Fase 1: Identificación (Cualquier servidor libre atiende) receive
  identificar(nombre, contrasena, j);

```

```

    if (comprobar(nombre, contrasena)) send respuestaid[j](tokensesion[i], listaCuentas); salir[i] :=
false;
    // Fase 2: Sesión Privada (Bucle de operaciones) while(salir[i] == false) // Operación
sin guarda: siempre se puede seleccionar cuenta if receive SeleccionarCuenta[i](token,
cuenta, j) ->seleccionada[i] = cuenta; tipo[i] = cuenta.TipoAcceso; send respuestaop[j](tipo[i]);
    // Operaciones con GUARDAS: Solo si hay cuenta seleccionada != 0
    // Consultar Saldo [] seleccionada[i] != 0; receive ConsultarSaldo[i](token, j)
->send respuestaop[j](seleccionada[i].saldo);
    // Ingresar Dinero (Requiere cuenta y permiso de escritura 'tipo') [] seleccionada[i]
!= 0 tipo[i]; receive IngresarDinero[i](token, cantidad, j) ->seleccionada[i].saldo
+= cantidad; send respuestaop[j](seleccionada[i].saldo);
    // Retirar Dinero (Uso de IF interno para control de saldo) [] seleccionada[i]
!= 0 tipo[i]; receive RetirarDinero[i](token, cantidad, j) ->if (cantidad <= seleccionada[i].saldo
seleccionada[i].saldo -= cantidad; send respuestaop[j](seleccionada[i].saldo); else send respuestaop[j](
    // Bloquear Cuenta [] seleccionada[i] != 0 tipo[i]; receive BloquearCuenta[i](token,
j) ->seleccionada[i] = 0; send respuestaop[j](resultado);
    // Salir Cliente: Cierra el bucle interno y limpia estado [] seleccionada[i] !=
0 tipo[i]; receive SalirCliente[i](token, j) ->salir[i] = true; send respuestaop[j](ok); selecci
0; tipo[i] = 0;

```

Ejercicios Tema 3

4.1. Exclusión Mutua de Anillo

```
1  Cliente[1..n] {
2      send(solicitar[i]);
3      receive(conceder[i]);
4      // Seccion critica
5      send(liberar[i]);
6      ...
7  }
8
9  Replica[1..n]{
10     ...
11     receive(token[i]);
12     if (solicitar.size() != 0){
13         receive(solicitar[i]);
14         send(conceder[i]);
15         receive(liberar[i]);
16     }
17     send(token[(i mod n) + 1]);
18 }
```

Listing 4.1: Algoritmo de Exclusión Mutua de Anillo

Kubernetes

5.1. ¿Por qué existe Kubernetes?

Kubernetes es una plataforma de código abierto que gestiona contenedores y automatiza sus recursos (almacenamiento, red y seguridad), asegurando que las aplicaciones funcionen siempre en el estado deseado.

A diferencia de las herramientas de infraestructura DevOps tradicionales que requieren una gestión manual, Kubernetes destaca por la reproducibilidad de la infraestructura. En el DevOps moderno, solucionar un problema de forma manual en un solo componente genera inconsistencias. Kubernetes evita esto automatizando la réplica exacta de los componentes en todo el sistema, lo que reduce los fallos y optimiza la administración en los centros de datos.

5.1.1. Repaso de algunos términos clave antes de empezar

El texto define los conceptos fundamentales para entender el ecosistema de Kubernetes:

5.1.2. Componentes de la Arquitectura

- **Control plane** (Plano de control): El “cerebro” del clúster (a veces denominado *Masters*). Se encarga de programar los contenedores y gestionar todos los objetos de Kubernetes.
- **Node** (Nodo): Una máquina que ejecuta el proceso kubelet.
- **kubelet**: El agente de Kubernetes que corre en los nodos del clúster. Hace lo que el plano de control le ordena.
- **kubectl**: La herramienta de línea de comandos para comunicarse con el plano de control de Kubernetes.

Contenedores y Despliegue

- **Container** (Contenedor): Una imagen Docker u OCI que típicamente ejecuta una aplicación.

- OCI (Open Container Initiative): El formato común de imagen para construir aplicaciones ejecutables y autónomas. También conocido como imágenes Docker.
- Pod: El objeto de Kubernetes que encapsula un contenedor en ejecución.
- Deployment (Despliegue): Una colección de Pods que es gestionada por Kubernetes.
- DaemonSet: Al igual que un *deployment*, pero se ejecuta en cada nodo de un clúster.

Interfaces

- CNI y CSI: Las interfaces de red y almacenamiento de contenedores (*Container Networking Interface* y *Container Storage Interface*, respectivamente) que permiten red y almacenamiento acoplables (*pluggable*) para los Pods que corren en Kubernetes.

5.1.3. El problema del desvío de infraestructura (Infrastructure Drift) y Kubernetes

La gestión de la infraestructura de manera reproducible implica controlar el *infrastructure drift* (desvío de configuración), el cual ocurre a medida que el hardware, las normativas de cumplimiento y los requisitos del centro de datos cambian con el tiempo. Esto afecta tanto a la definición de las aplicaciones como a la administración de los hosts donde se ejecutan.

Los ingenieros de TI suelen enfrentarse constantemente a tareas repetitivas y tediosas (*toil*) como:

- Actualizar la versión de Java en una flota entera de servidores.
- Asegurar que ciertas aplicaciones no se ejecuten en ubicaciones específicas.
- Reemplazar o escalar hardware antiguo o dañado, migrando las aplicaciones fuera de él.
- Gestionar manualmente las rutas de balanceo de carga.
- Olvidar documentar los nuevos cambios en la infraestructura debido a la falta de un lenguaje de configuración común y obligatorio.

A medida que se actualizan los servidores en la nube o en un centro de datos local, aumenta la probabilidad de que su estado real se desvíe de la arquitectura de TI planificada, provocando que las aplicaciones corran en lugares incorrectos, con asignaciones de recursos erróneas o con accesos a módulos de almacenamiento equivocados.

Soluciones y evolución tecnológica

Kubernetes permite centralizar la gestión de todo el espacio de estados de las aplicaciones mediante `kubectl`, un cliente de línea de comandos que realiza llamadas REST al servidor de la API de Kubernetes, o mediante clientes programáticos.

A diferencia de tecnologías previas como *Puppet*, *Chef*, *Mesos*, *Ansible* o *SaltStack*:

- Kubernetes hereda las capacidades de gestión de estado de herramientas como *Puppet* y los conceptos de programación de aplicaciones de *Mesos*.
- Mientras que *Ansible*, *SaltStack* y *Terraform* se encargan de la configuración inicial del sistema operativo (como cortafuegos o instalación de binarios), Kubernetes gestiona estos conceptos utilizando contenedores privilegiados en entornos Linux (conocidos como *HostProcess Pods* en Windows). Un ejemplo de esto es el componente kube-proxy, que maneja las reglas de iptables para enrutar el tráfico de red.

Finalmente, grandes compañías como Google, Microsoft, Amazon y VMware han adoptado la contenedorización como una estrategia clave para ejecutar miles de aplicaciones en entornos de nube y *bare metal*, convirtiendo a Kubernetes en el estándar moderno indiscutible para la orquestación de contenedores.

5.1.4. Contenedores e imágenes

Las aplicaciones tienen dependencias que deben ser cubiertas por el host donde se ejecutan. En la era previa a los contenedores, los desarrolladores lograban esto de manera improvisada (por ejemplo, una aplicación Java requería una JVM en ejecución junto con reglas de cortafuegos para comunicarse con una base de datos).

En su esencia, Docker puede entenderse como una forma de ejecutar contenedores, donde un contenedor es una imagen OCI (*Open Container Initiative*) en ejecución. La especificación OCI es un estándar para definir una imagen ejecutable por programas como Docker, y consiste básicamente en un archivo *tarball* con varias capas. Cada una de estas capas contiene elementos como binarios de Linux y archivos de la aplicación. Así, al ejecutar un contenedor, el *runtime* (como Docker, containerd o CRI-O) toma la imagen, la desempaqueta y arranca un proceso en el sistema host con el contenido de dicha imagen.

Los contenedores añaden una capa de aislamiento que elimina la necesidad de gestionar librerías en un servidor o precargar la infraestructura con otras dependencias accidentales. Por ejemplo, si se tienen dos aplicaciones Ruby que requieren versiones diferentes de la misma librería, se pueden utilizar dos contenedores independientes. Cada aplicación queda aislada con la versión específica que necesita. Esto soluciona la famosa frase: “*Bueno, en mi máquina funciona*”. El uso de imágenes simplifica la ejecución del mismo software en diferentes servidores.

Al combinar el uso de imágenes con Kubernetes se obtienen servidores inmutables y una simplicidad de nivel empresarial. Como los contenedores se están convirtiendo rápidamente en el estándar de la industria para el despliegue de software, vale la pena mencionar algunos datos:

- En una encuesta realizada a más de 88,000 desarrolladores, Docker y Kubernetes ocuparon el tercer lugar entre las tecnologías de desarrollo más queridas en 2020, justo detrás de Linux y Docker solo.
- Datadog descubrió recientemente que Docker abarca el 50% o más del flujo de trabajo del desarrollador promedio. Asimismo, la adopción a nivel corporativo supera el 25% de todas las empresas.

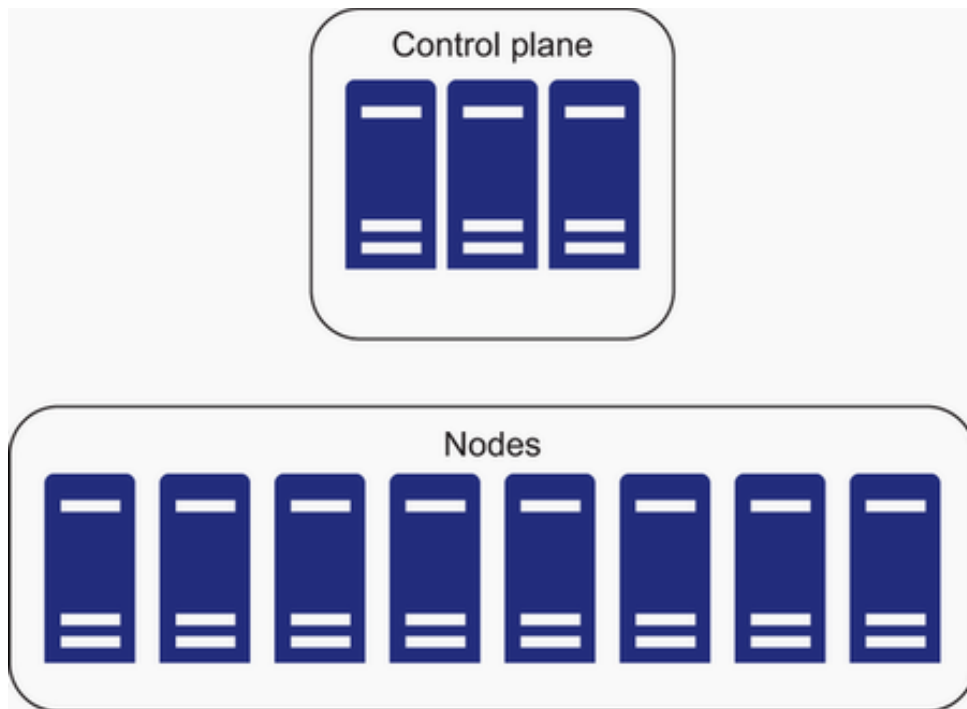


Figura 5.1: Aislamiento de dependencias mediante contenedores (Figura 1.1).

En conclusión, la necesidad de automatizar los contenedores es el lugar exacto donde encaja Kubernetes. Esta plataforma domina su sector de la misma manera que lo hicieron la base de datos Oracle y la plataforma de virtualización vSphere en sus mejores momentos. Se predice para Kubernetes una longevidad similar a la de esas tecnologías.

El propósito de este libro es llevarte más allá de los principios básicos hacia el núcleo de bajo nivel. Comencemos analizando un flujo de trabajo de Kubernetes extremadamente simplificado que demuestra algunos de los principios de nivel superior para construir y ejecutar microservicios.

5.1.5. Fundamentos esenciales de Kubernetes (Core foundation of Kubernetes)

En su núcleo, todo en Kubernetes se define mediante archivos de texto plano utilizando YAML o JSON. A través de este enfoque declarativo, la plataforma ejecuta imágenes OCI y permite configurar reglas de red, autenticación y autorización basada en roles (RBAC), entre otros recursos. Al aprender esta sintaxis única y su estructura, es posible configurar, gestionar y optimizar cualquier sistema de Kubernetes.

Flujo de trabajo y construcción de imágenes

Para ilustrar el despliegue de una arquitectura de microservicios básica (por ejemplo, un servidor MySQL y un cliente personalizado en Python), el proceso comienza con la definición de sus respectivos archivos Dockerfile:

```
# Servidor MySQL
FROM alpine:3.15.4
```

```

RUN apk add --no-cache mysql
ENTRYPOINT ["/usr/bin/mysqld"]

# Cliente personalizado en Python
FROM python:3.7
WORKDIR /myapp
COPY src/requirements.txt ./
RUN pip install -r requirements.txt
COPY src /myapp
CMD [ "python", "mysql-custom-client.py" ]

```

Típicamente, estas imágenes se construyen (docker build) y se suben (docker push) a un registro OCI (ya sea uno privado como *Harbor* o uno público como *Docker Hub*) para que los nodos del clúster puedan descargarlas durante el tiempo de ejecución.

Definición y despliegue en Kubernetes

Para ejecutar estas aplicaciones como contenedores dentro de Kubernetes, se encapsulan en objetos llamados Pods. La definición se realiza en un único archivo YAML (por ejemplo, `my-app.yaml`):

```

apiVersion: v1
kind: Pod
metadata:
  name: core-k8s
spec:
  containers:
    - name: my-mysql-server
      image: myregistry.com/mysql-server:v1.0
---
apiVersion: v1
kind: Pod
metadata:
  name: core-k8s-mysql
spec:
  containers:
    - name: my-sqlclient
      image: myregistry.com/mysql-custom-client:v1.0
      command: ['tail', '-f', '/dev/null']

```

Este archivo se envía al servidor de la API de Kubernetes ejecutando el comando `kubectl create -f my-app.yaml`. El plano de control almacena la definición y se encarga de que ambos Pods se programen y ejecuten en algún lugar del clúster.

Consistencia eventual y escalabilidad

Este proceso de despliegue no es instantáneo. Requiere que los nodos del clúster respondan a los eventos y actualicen su estado mediante el agente kubelet, el cual se comunica constantemente con la API. Debido a que las imágenes deben descargarse y pueden ocurrir fallos en el camino, Kubernetes se diseña bajo un modelo de consistencia eventual (*eventually consistent system*). En lugar de garantizar una consistencia inmediata, el sistema trabaja continuamente en la reconciliación para llevar el estado real del clúster hacia el estado deseado que el usuario ha solicitado.

Este modelo declarativo escala de forma natural a escenarios de producción complejos. Si se solicita desplegar cinco instancias de una aplicación distribuidas en tres zonas de disponibilidad de la nube, Kubernetes se encarga de la logística de la programación. Al delegar la implementación interna y el mantenimiento del estado a la propia plataforma, se elimina casi por completo la necesidad de realizar actualizaciones manuales o personalizadas con herramientas como *Ansible* o *Puppet* en entornos de producción.

- Gestión integral de la infraestructura mediante YAML: Kubernetes automatiza todos los aspectos de la pila tecnológica a través de su API, permitiendo definir las reglas tradicionales de TI como recursos de texto plano. Esto incluye:
 - Configuración de servidores, puertos y rutas IP.
 - Disponibilidad de almacenamiento persistente para las aplicaciones.
 - Alojamiento de *software* en servidores específicos o arbitrarios.
 - Aprovisionamiento de seguridad (como RBAC o reglas de red para la comunicación entre aplicaciones).
 - Configuración de DNS tanto a nivel global como por aplicación.

Todos estos componentes se representan como objetos dentro de la API de Kubernetes. La plataforma aplica los cambios, los monitorea y resuelve las interrupciones o fallos momentáneos de forma automática hasta alcanzar el estado final deseado. Al delegar esta automatización, se mitigan los incidentes imprevistos sin intervención manual directa, permitiendo que el equipo de DevOps se enfoque en resolver problemas complejos, planificar a futuro y encontrar soluciones óptimas para el negocio.

5.1.6. Características principales de Kubernetes (Kubernetes features)

Las plataformas de orquestación de contenedores permiten a los desarrolladores automatizar la ejecución de instancias, el aprovisionamiento de hosts y la extensión del ciclo de vida de las aplicaciones. Esencialmente, los contenedores necesitan Pods y los Pods necesitan a Kubernetes para cumplir con las siguientes funciones esenciales:

- API agnóstica de la nube: Expone una API unificada y neutral respecto al proveedor de nube a través del *API server*.

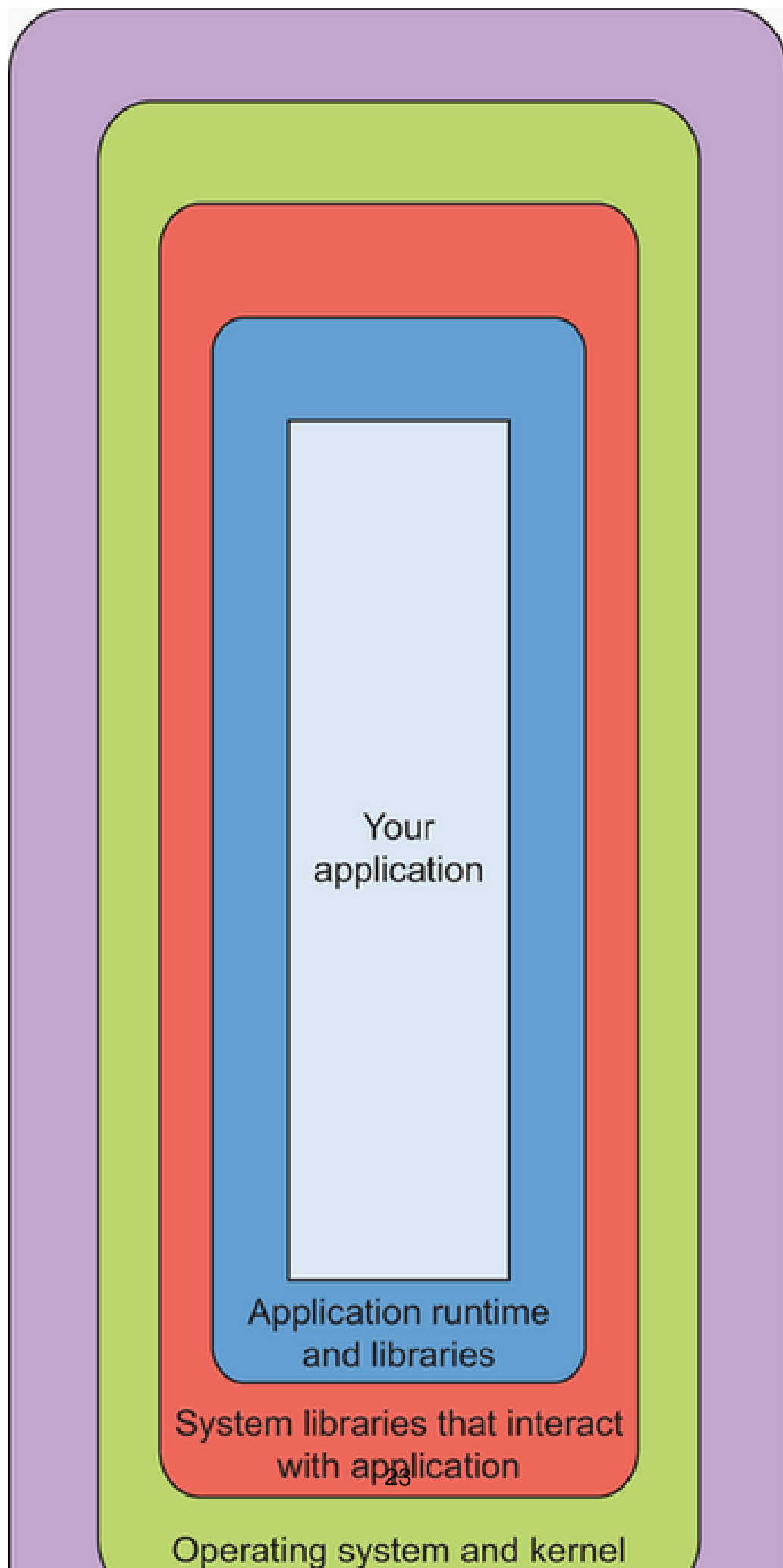
- Integración nativa: Se integra con las principales plataformas de nube e hipervisores mediante el *Kubernetes Controller Manager* (KCM).
- Tolerancia a fallos: Proporciona un marco resiliente para almacenar y definir el estado de los servicios, aplicaciones y configuraciones.
- Gestión de despliegues: Minimiza el tiempo de inactividad de cara al usuario durante las actualizaciones de hosts o aplicaciones mediante estrategias como los *rolling updates*.
- Escalado automático y balanceo de carga: Automatiza el escalado de hosts y aplicaciones, y crea integraciones de red internas y externas con balanceo de carga (*Service types: ClusterIP, NodePort o LoadBalancer*).
- Programación inteligente: El *Kubernetes scheduler* programa las aplicaciones en hardware virtualizado específico basándose en metadatos y etiquetas de nodo (*node labeling*).
- Alta disponibilidad y tareas por lotes: Garantiza que ciertos contenedores corran en todos los nodos mediante *DaemonSets* y permite ejecutar procesos por lotes (*Jobs*).
- Descubrimiento de servicios: Permite la resolución de nombres DNS mediante la integración nativa de *CoreDNS* con el servidor de la API.
- Extensibilidad de la API: Permite construir programas nativos basados en la API utilizando Definiciones de Recursos Personalizados (*CRDs*).
- Inspección y depuración: Facilita la auditoría de procesos fallidos y la ejecución remota dentro de los contenedores mediante comandos como `kubectl exec` y `kubectl describe`.
- Gestión de almacenamiento: Permite montar almacenamiento local o remoto y gestionar volúmenes declarativos con las API de *StorageClass* y *PersistentVolumes*.

La base de Kubernetes es un clúster compuesto por nodos. Aunque su complejidad suele ser una de las quejas más comunes entre los ingenieros, la plataforma resuelve un problema inherentemente complejo de estandarización y ciclo de vida que, de otro modo, sería muy difícil de solucionar.

Escenario típico de fallo y resiliencia

Si una organización no requiere alta disponibilidad, escalabilidad u orquestación, tal vez no necesite Kubernetes. Para entender su valor bajo presión, consideremos un escenario típico de fallo:

- a) Un nodo deja de responder al plano de control (*control plane*).
- b) El plano de control reprograma automáticamente los Pods que corrían en el nodo afectado hacia otros nodos operativos.



- c) Cuando un usuario consulta el estado mediante `kubectl`, el *API server* responde con la información corregida sobre el nodo caído y la nueva ubicación de los Pods.
- d) Todo el tráfico de los clientes hacia el servicio del Pod se redirige automáticamente a la nueva ubicación.
- e) Los volúmenes de almacenamiento persistente conectados a los Pods del nodo fallido se reasignan a la nueva ubicación del Pod para mantener la lectura de los datos previos.

El ecosistema y sistemas operativos inmutables

Bajo el capó, Kubernetes se apoya fuertemente en cientos de tecnologías de la pila de Linux. Por ello, resulta natural ejecutar Kubernetes sobre sistemas operativos inmutables, donde el SO base solo se actualiza por completo y en bloque, aportando ventajas críticas de estabilidad.

Kubernetes ofrece la flexibilidad de ejecutarse en la nube, en servidores físicos (*bare metal*), en una *Raspberry Pi*, en superordenadores/mainframes de última generación (como IBM PowerPC) e incluso en entornos críticos de defensa aeroespacial. A medida que el ecosistema *cloud native* madura, permite a las organizaciones adoptar mejores prácticas, prevenir problemas de manera proactiva y mantener la consistencia del entorno para evitar el *drift* (desvío de configuración) tecnológico.

5.1.7. Componentes y arquitectura de Kubernetes (Kubernetes components and architecture)

A alto nivel, la arquitectura de Kubernetes está compuesta por la infraestructura física o virtual (el hardware) y la división de este hardware para ejecutar tanto el plano de control como los nodos de trabajo (*worker nodes*):

- Infraestructura de hardware (Hardware infrastructure): Incluye las computadoras (servidores), la infraestructura de red, los sistemas de almacenamiento y el registro de contenedores (*container registry*).
- Nodos de trabajo de Kubernetes (Kubernetes worker nodes): Representan la unidad básica de cómputo dentro de un clúster de Kubernetes, encargados de ejecutar los contenedores de las aplicaciones.
- Plano de control de Kubernetes (Kubernetes control plane): Es la “nave nodriza” de Kubernetes. Este componente engloba el servidor de la API (*API server*), el programador (*scheduler*), el administrador de controladores (*controller manager*) y otros controladores internos encargados de mantener el estado global del sistema.

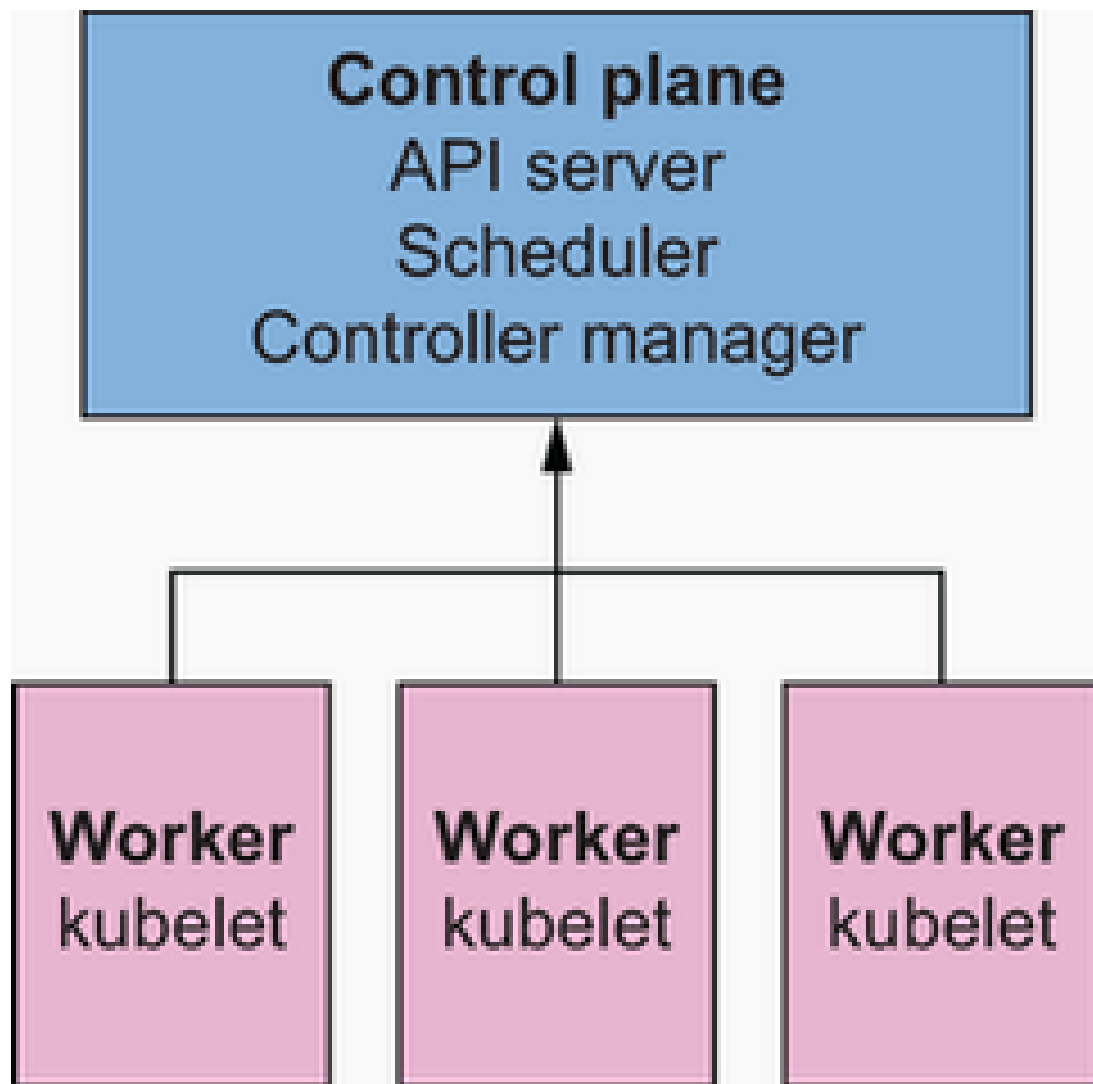


Figura 5.3: Arquitectura general de Kubernetes: Infraestructura, Plano de Control y Nodos de Trabajo (Figura 1.3).

5.1.8. La API de Kubernetes (The Kubernetes API)

El aspecto fundamental para administrar microservices y aplicaciones contenedorizadas en Kubernetes consiste simplemente en declarar objetos de su API; la plataforma se encarga de gestionar el resto de forma automática. Casi cualquier acción ejecutada a través de `kubectl` se traduce en una operación de lectura o escritura en un objeto definido y versionado dentro del servidor de la API (*API server*), el cual almacena los datos en su componente de persistencia, `etcd`.

El `kube-apiserver` expone una interfaz RESTful que permite realizar operaciones CRUD (crear, leer, actualizar y eliminar) sobre todos los recursos del sistema. Por lo general, todo objeto de la API de Kubernetes cuenta con tres elementos esenciales:

- Un nombre de versión de la API (*API version*, por ejemplo: `v1` o `rbac.authorization.k8s.io`).
- Un tipo de recurso (*kind*, por ejemplo: `kind: Deployment`).
- Una sección de metadatos (*metadata*).

Este robusto esquema de versionado de API (diseñado originalmente por Brian Grant, uno de los fundadores de Kubernetes) facilita las actualizaciones del sistema y define contratos claros para el ciclo de vida de los recursos en sus fases *Alpha*, *Beta* y *GA* (Disponibilidad General).

Tipos de recursos y Namespaces

En un clúster estándar de Kubernetes existen alrededor de 70 tipos de API distintos. Estos recursos se pueden consultar mediante el comando `kubectl api-resources`, el cual devuelve un listado con la siguiente estructura:

```
$ kubectl api-resources | head
NAME                SHORTNAMES  NAMESPACED  KIND
bindings            cs          false       Binding
componentstatuses   cs          false       ComponentStatus
configmaps           cm          true        ConfigMap
endpoints            ep          true        Endpoint
events              ev          true        Event
limitranges          limits      true        LimitRange
namespaces           ns          false       Namespace
nodes               no          false       Node
persistentvolumeclaims pvc         true        PersistentVolumeClaim
```

Como se observa en la salida, cada recurso de la API posee un nombre corto (*short name*), un nombre completo y un indicador booleano (*NAMESPACED*) que determina si el objeto está restringido o no a un Namespace.

Los Namespaces (Espacios de nombres) proporcionan una agrupación jerárquica y lógica para los recursos. Al desplegar una aplicación compuesta por múltiples microservicios, es una práctica común crear todos sus Pods, Services y *PersistentVolumeClaims* (PVCs) dentro de un mismo Namespace. Esto simplifica el aislamiento y permite que, al momento

de retirar la aplicación, baste con eliminar dicho Namespace para limpiar de forma limpia y ordenada todos los objetos asociados.

5.1.9. Ejemplo uno: Un minorista en línea (Example one: An online retailer)

Imaginemos un gran comercio minorista en línea que necesita escalar rápidamente para responder a la demanda estacional, como ocurre durante las vacaciones o festividades. Históricamente, predecir la demanda y escalar la infraestructura para soportarla ha sido uno de sus mayores desafíos.

Kubernetes resuelve la multitud de problemas asociados a la ejecución de sistemas distribuidos, altamente disponibles y escalables. No solo representa una mejor estrategia para operar el negocio, sino que constituye la plataforma más eficiente y efectiva para la gestión de sistemas.

Al combinar el potencial de Kubernetes con los proveedores de servicios en la nube, es posible ejecutar cargas de trabajo en servidores de terceros únicamente cuando se requieren recursos adicionales. Esto elimina por completo la necesidad de comprar, configurar y mantener hardware costoso que de otro modo permanecería inactivo la mayor parte del año.

5.1.10. Ejemplo dos: Una solución de donaciones en línea (Example two: An online giving solution)

Consideremos el caso de un sitio web de donaciones en línea que permite realizar contribuciones a diversas organizaciones benéficas. La plataforma comenzó como un sitio básico en *WordPress*, pero el crecimiento del negocio la llevó a evolucionar hacia una arquitectura compleja dependiente de *frameworks* de la JVM (como *Grails*), con una interfaz de usuario personalizada, una capa intermedia y una base de datos propia. Con el tiempo, los requisitos tecnológicos se dispararon hasta incluir aprendizaje automático, servidores de anuncios, mensajería y una enorme variedad de tecnologías (*Python*, *Lua*, *NGINX*, *PHP*, *MySQL*, *Cassandra*, *Redis*, *Elastic*, *ActiveMQ*, *Spark*, entre otras).

De máquinas virtuales a la orquestación nativa

La infraestructura inicial consistía en máquinas virtuales (VM) en la nube configuradas manualmente mediante *Puppet*. A medida que la empresa crecía, el diseño para escalar implicó añadir más y más VMs que solo alojaban una o dos aplicaciones.

La decisión de migrar a Kubernetes transformó radicalmente su operación:

- Reducción de costos y recursos: El número de máquinas virtuales se redujo drásticamente de aproximadamente 30 a solo 5, logrando además un escalado mucho más eficiente.
- Eliminación de la gestión manual: Se eliminó por completo el uso de *Puppet* y la configuración manual de servidores, delegando la administración de la infraestructura a los mecanismos automatizados de Kubernetes.

- Resolución de problemas de red y administración: La migración resolvió toda la categoría de problemas asociados a la administración de VMs y eliminó la complejidad de gestionar manualmente el DNS para la publicación de servicios complejos.
- Resiliencia predecible: Los tiempos de recuperación ante fallos catastróficos se volvieron mucho más predecibles y controlables desde la perspectiva de la infraestructura.

Este caso demuestra cómo adoptar una metodología estandarizada y dirigida por APIs permite realizar cambios masivos con rapidez, evidenciando el verdadero valor de la naturaleza declarativa de Kubernetes y su enfoque *cloud-native* para la orquestación de contenedores.

5.1.11. Cuándo no utilizar Kubernetes (When not to use Kubernetes)

A pesar de sus múltiples ventajas, existen ciertos casos de uso específicos donde Kubernetes puede no ser la opción más adecuada:

- Computación de alto rendimiento (HPC - High-Performance Computing): El uso de contenedores añade una capa adicional de abstracción y complejidad que puede penalizar el rendimiento. Aunque la latencia introducida por los contenedores es cada vez menor, si la aplicación es extremadamente sensible a variaciones de nano o microsegundos, Kubernetes no es la mejor alternativa.
- Sistemas heredados (*Legacy*): Ciertas aplicaciones antiguas tienen requisitos estrictos de hardware, *software* o latencia que dificultan enormemente su contenedorización. Un ejemplo común son los desarrollos comerciales adquiridos a terceros cuyos proveedores no ofrecen soporte oficial para ejecutarse dentro de contenedores o clústeres de Kubernetes.
- Migraciones rígidas: Existen implementaciones de sistemas heredados tan rígidas que migrarlas a Kubernetes no aporta ninguna ventaja competitiva real, más allá de la mera etiqueta de “estar contruidos sobre Kubernetes”. Sin embargo, cabe destacar que los mayores beneficios suelen percibirse tras la migración, cuando una aplicación monolítica se descompone en componentes lógicos capaces de escalar de forma independiente.

Lo fundamental en este punto es aprender y dominar los conceptos básicos, ya que Kubernetes resuelve la gran mayoría de los problemas de infraestructura modernos de una manera estable y eficiente en costos.

5.1.12. Resumen del capítulo (Summary)

En conclusión, Kubernetes simplifica la gestión de infraestructuras complejas mediante los siguientes pilares clave:

- Agnóstico de la infraestructura: La plataforma Kubernetes puede ejecutarse sobre cualquier tipo de entorno (nube, servidores físicos o *edge computing*).
- Ecosistema integrado y resiliente: Construye un ecosistema de componentes que trabajan en conjunto, lo que permite a las empresas prevenir fallos, recuperarse ante desastres y escalar en tiempo real cuando se requieren cambios urgentes.
- Control centralizado: Prácticamente cualquier acción en Kubernetes se puede realizar utilizando una única herramienta de línea de comandos: `kubectl`.
- Orquestación completa: Transforma uno o más ordenadores en un clúster unificado que proporciona una plataforma para desplegar y alojar contenedores, ofreciendo orquestación, gestión de almacenamiento y redes distribuidas.
- Evolución tecnológica: Nace como la evolución natural de enfoques previos orientados a la configuración y a la gestión guiada por contenedores.
- El Pod como unidad fundamental: El *Pod* es el bloque de construcción básico en Kubernetes y el elemento central que da soporte a todas sus características, incluyendo el escalado, la alta disponibilidad, la resolución de nombres por DNS y las reglas de seguridad RBAC.
- Gestión declarativa orientada a APIs: El ciclo de vida completo de las aplicaciones se administra de manera sencilla realizando llamadas declarativas al servidor de la API de Kubernetes (*API server*).

5.2. ¿Por qué el Pod? (Why the Pod?)

Este capítulo cubre los siguientes puntos:

- ¿Qué es un Pod?
- Ejemplo de una aplicación web y por qué necesitamos el Pod.
- Cómo está construido Kubernetes para soportar los Pods.
- El plano de control de Kubernetes.

En el capítulo anterior, proporcionamos una visión general a alto nivel de Kubernetes e introducimos sus características, componentes principales y arquitectura. También mostramos un par de casos de uso empresarial y esbozamos algunas definiciones sobre contenedores. La abstracción del *Pod* de Kubernetes para ejecutar miles de contenedores de manera flexible ha sido una parte fundamental en la transición hacia los contenedores en las empresas. En este capítulo, abordaremos el Pod y cómo Kubernetes fue diseñado para soportarlo como el bloque de construcción básico de las aplicaciones.

Como se mencionó brevemente en el capítulo 1, un Pod es un objeto que se define dentro de la API de Kubernetes, al igual que la mayoría de las cosas en esta plataforma. El Pod es la unidad atómica más pequeña que se puede desplegar en un clúster de Kubernetes, y todo el sistema está construido en torno a su definición. El Pod permite definir

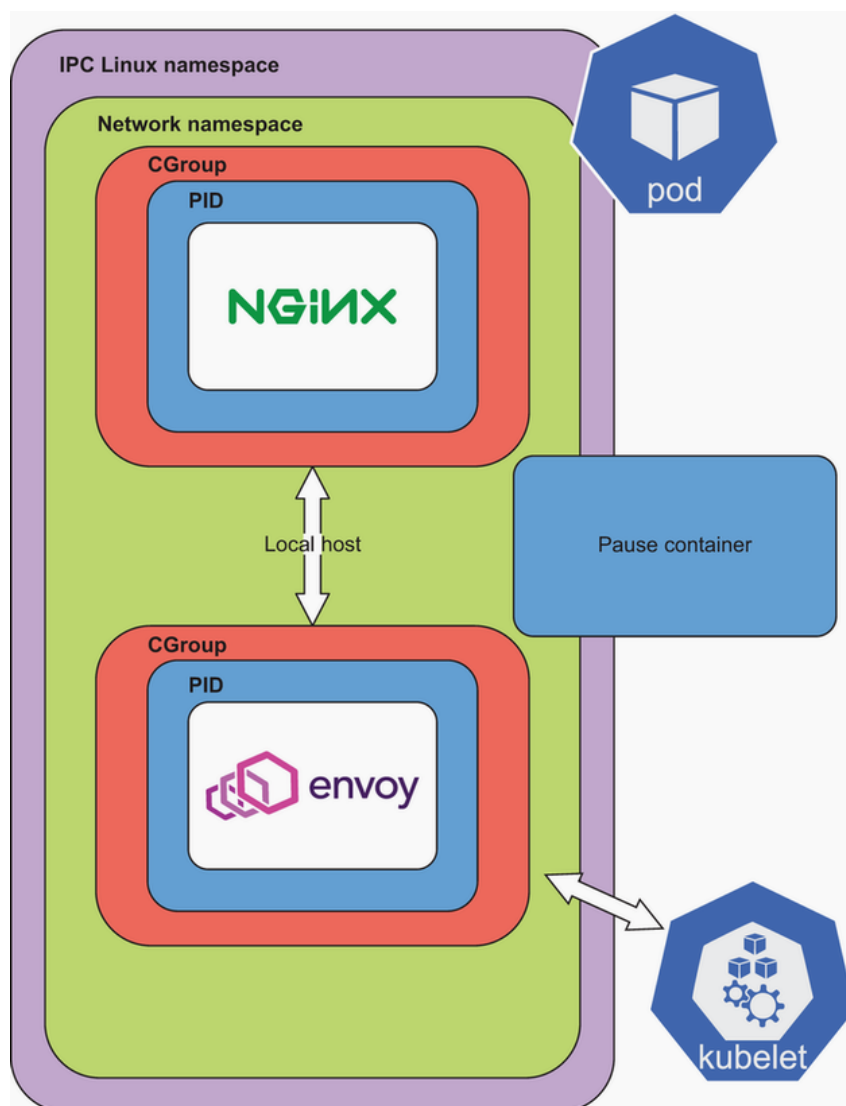


Figura 5.4: El Pod como abstracción que encapsula uno o más contenedores (Figura 2.1).

un objeto que puede incluir múltiples contenedores, lo que faculta a Kubernetes a crear uno o más contenedores alojados conjuntamente en un nodo.

Muchos otros objetos de la API de Kubernetes utilizan Pods directamente o son objetos diseñados para darles soporte. Por ejemplo, un objeto de tipo *Deployment* utiliza Pods, al igual que los *StatefulSets* y los *DaemonSets*. Varios controladores de alto nivel de Kubernetes se encargan de crear y gestionar el ciclo de vida de los Pods. Los controladores son componentes de *software* que se ejecutan en el plano de control; entre los ejemplos de controladores integrados se encuentran el *controller manager*, el *cloud manager* y el *scheduler*.

Antes de profundizar en ello, hagamos una pequeña digresión analizando una aplicación web para luego vincularla nuevamente con Kubernetes, el Pod y el plano de control.

Nota

Es posible que note que utilizamos el término “plano de control” (control plane) para definir al grupo de nodos que ejecutan el controlador, el administrador de controladores y el programador. En ocasiones también se les denomina “maestros” (masters), pero en este libro utilizaremos exclusivamente “plano de control” al referirnos a estos componentes.

5.2.1. Ejemplo de una aplicación web (An example web application)

Para comprender la necesidad de existencia del Pod y cómo Kubernetes soporta las aplicaciones contenedorizadas, analizaremos el caso de la empresa de bebidas energéticas *Zeus Zap*. Su sitio web de comercio electrónico consta de tres capas: interfaz de usuario (UI), capa intermedia (microservicios) y base de datos en el *backend*.

Un desglose de una sección de su infraestructura tecnológica (Figura 2.2) incluye:

- Un *frontend* en JavaScript servido por *NGINX*.
- Dos capas de controladores web que son microservicios en *Python* desarrollados con *Django*.
- Una base de datos *CockroachDB* en el *backend* que escucha en el puerto 6379, respaldada por almacenamiento persistente.

En un entorno de producción tradicional basado únicamente en contenedores independientes, el inicio de la aplicación se ejecutaría mediante los siguientes comandos de Docker:

```
$ docker run -t -i ui -p 80:80
$ docker run -t -i miroservice-a -p 8080:8080
$ docker run -t -i miroservice-b -p 8081:8081
$ docker run -t -i cockroach:cockroach -p 6379:6379
```

Limitaciones del enfoque tradicional y nuevos requisitos

Al poner en marcha estos servicios bajo este esquema, la compañía rápidamente se enfrenta a varios problemas de diseño y arquitectura de infraestructura:

- a) Conflictos de puertos: No es posible ejecutar múltiples copias del contenedor de la UI a menos que se coloque un balanceador de carga externo frente al puerto 80, ya que solo existe un puerto 80 disponible en la máquina host.
- b) Rigidez de red: No se puede migrar el contenedor de la base de datos (*CockroachDB*) a otro servidor a menos que se modifique e inyecte dinámicamente la nueva dirección IP en la aplicación web (o se implemente un servidor DNS que se actualice dinámicamente con los movimientos del contenedor).
- c) Restricciones de alta disponibilidad: Cada instancia de la base de datos debe ejecutarse en un servidor físico diferente para garantizar una tolerancia a fallos real.
- d) Persistencia y recuperación volátil: Si una instancia de la base de datos falla, se requiere un mecanismo para mover sus datos a un nuevo nodo y reclamar el espacio de almacenamiento no utilizado en el host anterior.

A raíz de esto, surgen requisitos fundamentales que una plataforma de orquestación de contenedores debe resolver:

- Red compartida: Permitir que cientos de procesos compartan la red de manera inteligente sin generar conflictos al enlazarse a los mismos puertos.
- Desacoplamiento de almacenamiento: Migrar y separar los volúmenes de datos de los binarios de la aplicación, evitando ensuciar los discos locales de los servidores.
- Optimización de recursos: Maximizar el uso de la CPU y la memoria disponibles para reducir costos operativos.

Nota: El fenómeno del vecino ruidoso (*Noisy Neighbor*)

Ejecutar múltiples procesos concurrentes en un mismo servidor suele provocar que las aplicaciones compitan de forma agresiva por recursos escasos (CPU, memoria). Un sistema de orquestación eficiente debe mitigar este comportamiento para asegurar un rendimiento predecible.

Por lo tanto, gestionar aplicaciones contenedorizadas a gran o pequeña escala exige un mayor nivel de control en los siguientes aspectos:

- Planificación consciente del almacenamiento (*Storage-aware scheduling*): Capacidad de programar la ejecución de un proceso en estricta coordinación con la disponibilidad de sus datos.
- Balanceo de carga consciente del servicio (*Service-aware network load balancing*): Redirigir el tráfico hacia las nuevas direcciones IP de forma transparente a medida que los contenedores migran de una máquina a otra.

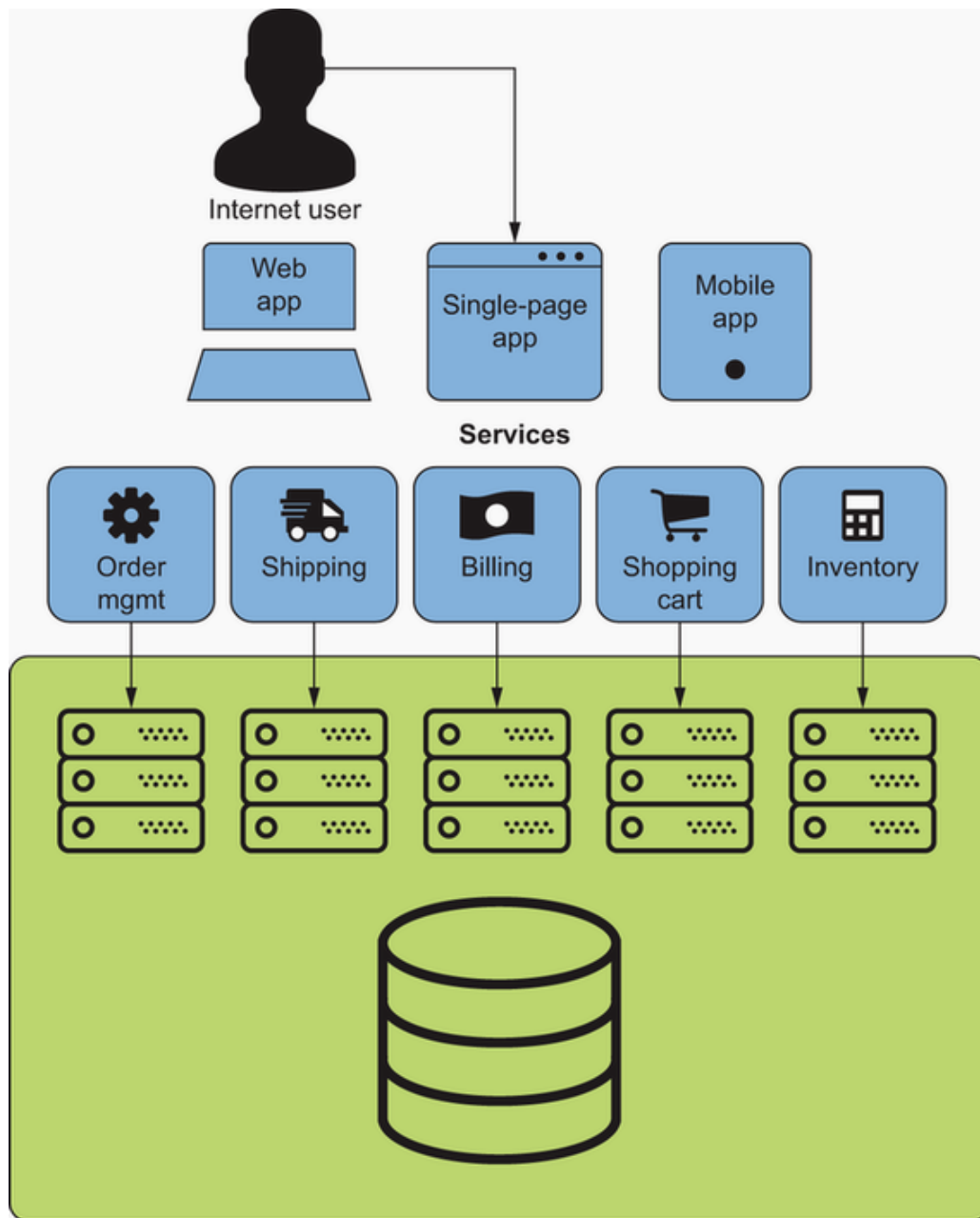


Figura 5.5: Arquitectura de microservicios de Zeus Zap y sus desafíos de red y almacenamiento (Figura 2.2).

Estas revelaciones tecnológicas coinciden con las mismas necesidades que dieron origen a las herramientas de orquestación y planificación distribuida de la década de 2000, tales como *Borg* (el sistema interno de orquestación de contenedores de Google) y *Mesos* (una aplicación de código abierto), ambos predecesores históricos de Kubernetes.

5.2.2. Infraestructura para nuestra aplicación web (Infrastructure for our web application)

Sin un *software* de orquestación de contenedores como Kubernetes, las organizaciones requieren una gran cantidad de componentes independientes en su infraestructura. Para ejecutar una aplicación tradicional, se necesitan múltiples máquinas virtuales (VM) en la nube o servidores físicos, además de identificadores estables (como IPs estáticas o registros DNS) para localizar los servicios.

Las cargas de trabajo de los servidores varían significativamente según su propósito:

- Optimización de hardware: Las bases de datos suelen requerir servidores con mucha memoria RAM y almacenamiento de baja latencia (como SSDs de alto rendimiento). Por el contrario, los microservicios pueden demandar más capacidad de cómputo (CPUs) pero menos memoria.
- Almacenamiento diferenciado: Las copias de seguridad o las aplicaciones que procesan datos en memoria y rara vez tocan el disco pueden utilizar un almacenamiento mucho más lento y económico.
- Gestión de accesos y seguridad: Los servidores de integración continua (como *Jenkins* o *CircleCI*) exigen un acceso completo de escritura a los servidores, mientras que los sistemas de monitoreo solo requieren acceso de lectura para recolectar métricas. A esto se le debe sumar la gestión de autenticación y autorización para el personal humano.

En resumen, una plataforma de despliegue propia exige aprovisionar de forma independiente: una plataforma base (VM o servidores físicos), balanceadores de carga, sistemas de descubrimiento de aplicaciones, soluciones de almacenamiento y sistemas de seguridad.

Para sostener este ecosistema operativo, el equipo de DevOps se ve obligado a mantener (además de muchos otros subsistemas):

- Sistemas de registro centralizado (*logging*).
- Herramientas de monitoreo, alertas y métricas.
- Flujos de integración y entrega continua (CI/CD).
- RespalDOS de seguridad (*backups*).
- Gestión de secretos y credenciales.

A diferencia de estas plataformas caseras de entrega de aplicaciones, Kubernetes ofrece ventajas inmediatas de forma nativa (*out of the box*), tales como rotación de registros, herramientas de inspección y gestión integrada de logs, lo que reduce drásticamente la sobrecarga operativa antes de enfrentarse a los verdaderos desafíos del negocio.

5.2.3. Requisitos operacionales (Operational requirements)

A diferencia de los minoristas tradicionales, la empresa de bebidas energéticas *Zeus Zap* no experimenta un crecimiento estacional típico, sino que enfrenta picos masivos de demanda vinculados a su patrocinio de eventos de deportes electrónicos (*e-sports*). Las campañas de marketing y las promociones en directo junto a *streamers* generan uno de los patrones de tráfico más difíciles de gestionar para el equipo de DevOps: el tráfico de ráfaga (*burst traffic*).

Debido al impacto de las redes sociales durante estos eventos, la empresa está sumamente preocupada por las caídas del sistema, ya que los costos asociados al tiempo de inactividad (*downtime*) son devastadores:

- Costo económico: De acuerdo con un estudio de Gartner, el costo promedio de la inactividad de TI se estima en \$5,600 dólares por minuto. Una interrupción de apenas dos horas puede traducirse en una pérdida promedio de \$672,000 dólares.
- Costo humano: Los fallos constantes en la infraestructura desgastan al personal técnico. El agotamiento laboral (*burnout*) de los empleados cuesta a las industrias estadounidenses entre \$125 y \$190 mil millones de dólares anuales.

Para mitigar esto, la mayoría de las empresas exigen altos niveles de disponibilidad y redundancia, tanto a nivel de *software* como de hardware, además de mecanismos para revertir cambios (*rollbacks*). Sin embargo, para mantener los presupuestos bajo control, estas mismas organizaciones necesitan reducir la infraestructura durante los períodos de menor demanda.

Por lo tanto, la gestión de costos suele entrar en conflicto directo con los requisitos de disponibilidad del negocio. En resumen, una aplicación web moderna requiere equilibrar de manera eficiente cuatro pilares esenciales:

- a) Escalabilidad: Capacidad de responder a ráfagas imprevistas de tráfico.
- b) Alta disponibilidad: Redundancia para evitar puntos únicos de fallo.
- c) Control de versiones: Facilitar despliegues seguros y actualizaciones reversibles.
- d) Gestión de costos: Optimización y dimensionamiento elástico de los recursos.

5.2.4. El Pod como unidad de despliegue (The Pod as a deployment unit)

A grandes rasgos, un Pod está compuesto por una o más imágenes OCI que se ejecutan como contenedores en un nodo de un clúster de Kubernetes. El nodo es una unidad de cómputo (un servidor) que ejecuta el agente kubelet y que, al igual que los demás componentes de la plataforma, se representa como un objeto de la API.

Desplegar un Pod de forma directa es tan sencillo como crear su manifiesto y enviarlo al clúster mediante la línea de comandos:

```
$ cat << EOF > pod.yaml
apiVersion: v1                                # 1. Versión de la API en el servidor
kind: Pod                                     # 2. Tipo de objeto de la API
metadata:
  name: busybox
spec:
  containers:
    - name: busybox
      image: mycontainerregistry.io/foo      # 3. Nombre de la imagen en el registro
EOF
$ kubectl create -f pod.yaml                  # 4. Comando de ejecución de kubectl
```

La herramienta `kubectl` actúa como la interfaz de línea de comandos principal para interactuar mediante peticiones REST con el servidor de la API de Kubernetes. Al arrancar el Pod, se puede verificar su ejecución dentro del espacio de nombres por defecto utilizando el comando `kubectl get po`.

Abstracciones de alto nivel

En la práctica, los Pods rara vez se despliegan de forma directa o aislada. En su lugar, el usuario define objetos de la API de nivel superior que se encargan de crear y gestionar el ciclo de vida de los Pods de forma automatizada:

- **Deployments:** El objeto más utilizado en Kubernetes; es el estándar para desplegar aplicaciones sin estado (*stateless*) como los microservicios.
- **Jobs:** Ejecutan Pods diseñados para realizar tareas por lotes (*batch processes*) que finalizan tras completar su función.
- **StatefulSets:** Diseñados para alojar aplicaciones con estado (*stateful*) como las bases de datos. Sus características principales incluyen:
 - Nombramiento ordinal de Pods para garantizar identificadores de red únicos y estables.
 - Almacenamiento persistente vinculado de forma estricta siempre al mismo Pod.
 - Mecanismos ordenados para el arranque, escalado y actualización de las instancias.
- **DaemonSets:** Se utilizan para garantizar que una copia de un Pod específico se ejecute como un “agente” en cada uno de los nodos del clúster (común para servicios del sistema como redes, almacenamiento o recolección de *logs*).

Consejo práctico: Evitar la etiqueta `latest` en producción

Los registros de contenedores permiten etiquetar imágenes con el tag `latest`. Se desaconseja totalmente su uso en entornos de producción, ya que este puntero no es inmutable y puede provocar fallos catastróficos al descargar inadvertidamente versiones

de software no probadas. La mejor práctica exige utilizar siempre etiquetas con versiones específicas o, idealmente, el SHA (hash) de la imagen, el cual sí garantiza una inmutabilidad real.

El flujo de la aplicación Zeus Zap

Para la aplicación web de *Zeus Zap*, en lugar de depender de configuraciones locales que disparen comandos individuales de Docker (`docker run`) al iniciar un servidor, se utilizan abstracciones como *Deployments* y *StatefulSets*. Estos controladores de alto nivel generan objetos de réplica (*ReplicaSets*), los cuales se encargan finalmente de instanciar y empaquetar los binarios y dependencias dentro de los Pods (Figura 2.3).

5.2.5. Espacios de nombres de Linux vs. Kubernetes (A bunch of Linux namespaces)

Es fundamental no confundir los Namespaces de Kubernetes (aquellos creados con `kubectl create ns`) con los namespaces de Linux. Mientras que los primeros son abstracciones lógicas de la API para organizar recursos, los namespaces de Linux son una característica nativa del kernel que permite el aislamiento y la separación de procesos.

A nivel elemental, un Pod es simplemente un conjunto de namespaces de Linux configurados de una forma específica. Un Pod comparte y encapsula los siguientes namespaces del kernel:

- Uno o más namespaces de PID (identificadores de procesos).
- Un único namespace de red (*networking*).
- Namespace de IPC (comunicación entre procesos).
- Namespace de cgroup (grupos de control para el límite de recursos).
- Namespace de mnt (puntos de montaje del sistema de archivos).
- Namespace de user (identificadores de usuario y grupo).

Estos namespaces de Linux actúan como componentes del sistema de archivos del kernel que proveen la funcionalidad base para tomar una imagen y transformarla en un contenedor en ejecución.

Relación con los requisitos de la aplicación

¿Por qué es importante entender esto? Si volvemos a los requisitos operativos de la aplicación web de *Zeus Zap*, la capacidad de escalar es vital. El Pod no solo nos otorga la facultad de desplegar contenedores, sino que permite escalar la infraestructura para manejar picos de tráfico. Para reducir costos y escalar verticalmente de manera eficiente, es imprescindible tener la capacidad de ajustar de forma aislada los límites de recursos de un contenedor (mediante el namespace de *cgroup*).

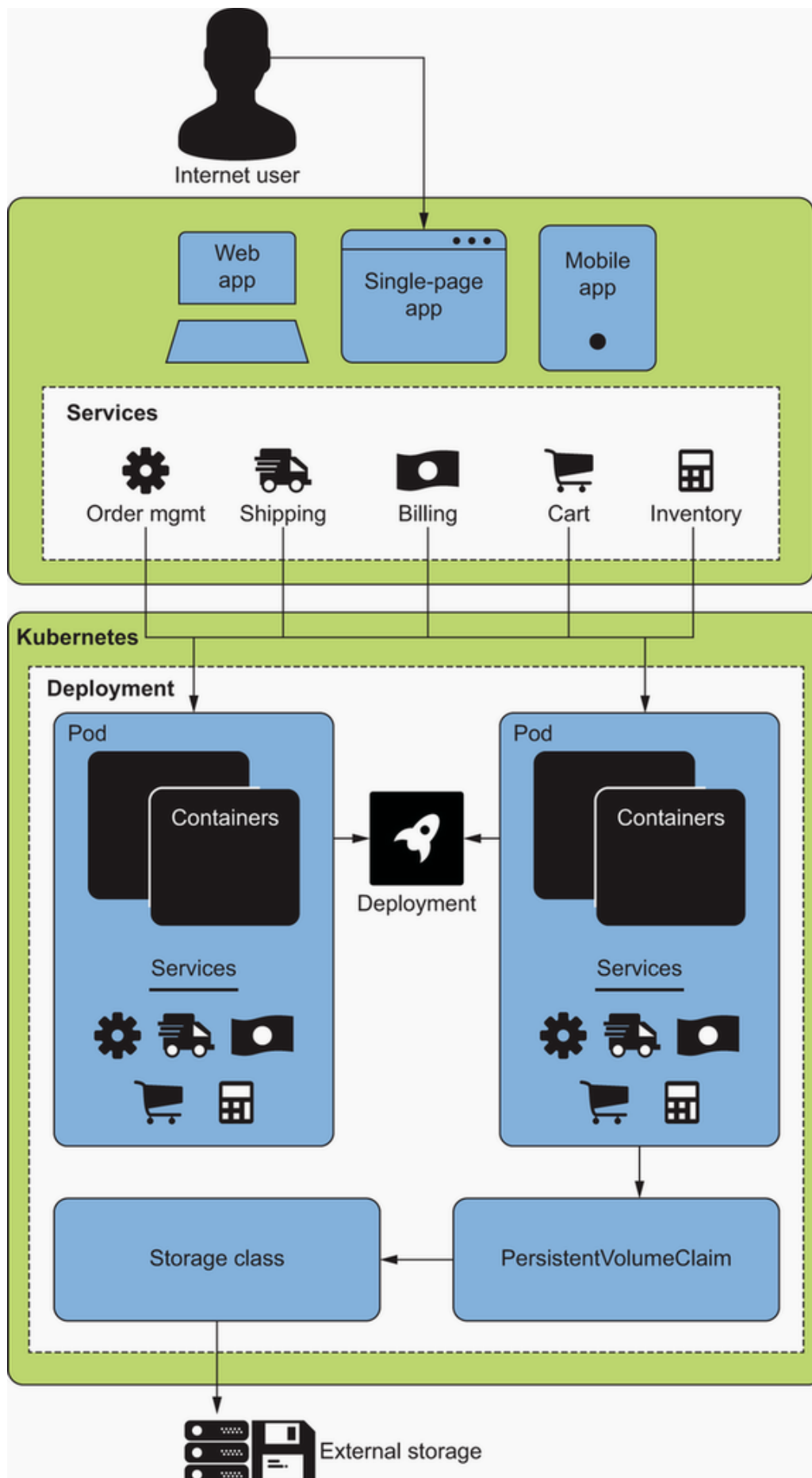


Figura 5.6: Controladores de alto nivel (Deployments/StatefulSets) gestionando el ciclo de vida de los Pods de Zeus Zap (Figura 33).

Asimismo, para que los microservicios de la empresa se comuniquen de forma fiable con el servidor de *CockroachDB*, se requiere un sistema definitivo de red y resolución de servicios. El Pod -y su base en los namespaces de Linux- proporciona el soporte para todas estas características.

Dentro del namespace de red del Pod existe una pila de red virtualizada que se conecta directamente a un sistema de Red Definida por Software (SDN) que se extiende a lo largo de todo el clúster de Kubernetes. De este modo, la necesidad de escalar mediante el balanceo de carga entre múltiples Pods se ve totalmente respaldada por este *framework* de red unificado.

5.2.6. Kubernetes, la infraestructura y el Pod (Kubernetes, infrastructure, and the Pod)

Dentro de Kubernetes, una unidad de potencia de cómputo (CPU y memoria) se representa mediante un objeto de la API denominado Node (Nodo). Un nodo es simplemente un servidor que ejecuta un conjunto de componentes definidos. Los requisitos esenciales de un nodo incluyen:

- Un servidor (físico o virtual).
- Un sistema operativo instalado (con soporte para distribuciones de Linux o Windows).
- systemd: El administrador de sistema y servicios de Linux.
- El kubelet: El agente principal del nodo.
- El entorno de ejecución de contenedores (*Container Runtime*, como Docker Engine, CRI-O o LXC).
- kube-proxy: El proxy de red que gestiona los servicios de Kubernetes.
- Un proveedor de CNI (*Container Network Interface*).

Un nodo puede ejecutarse en plataformas tan diversas como una *Raspberry Pi* o una máquina virtual en la nube.

El rol crítico del Kubelet

El kubelet es un programa binario que se ejecuta como un agente en cada nodo y se comunica constantemente con el *API server* a través de múltiples bucles de control (*control loops*). Sin este componente, un nodo no puede programar tareas ni considerarse parte del clúster. Comprender el kubelet es clave para diagnosticar problemas de bajo nivel (por ejemplo, por qué un nodo no se une al clúster o por qué un Pod no se despliega).

El kubelet garantiza lo siguiente:

- Que cualquier Pod programado en su host se ejecute correctamente mediante la vigilancia continua del estado del clúster.
- Informar al *API server* que el nodo está saludable a través de un mecanismo de latido (*heartbeat*), el cual se gestiona en el espacio de nombres kube-node-lease.

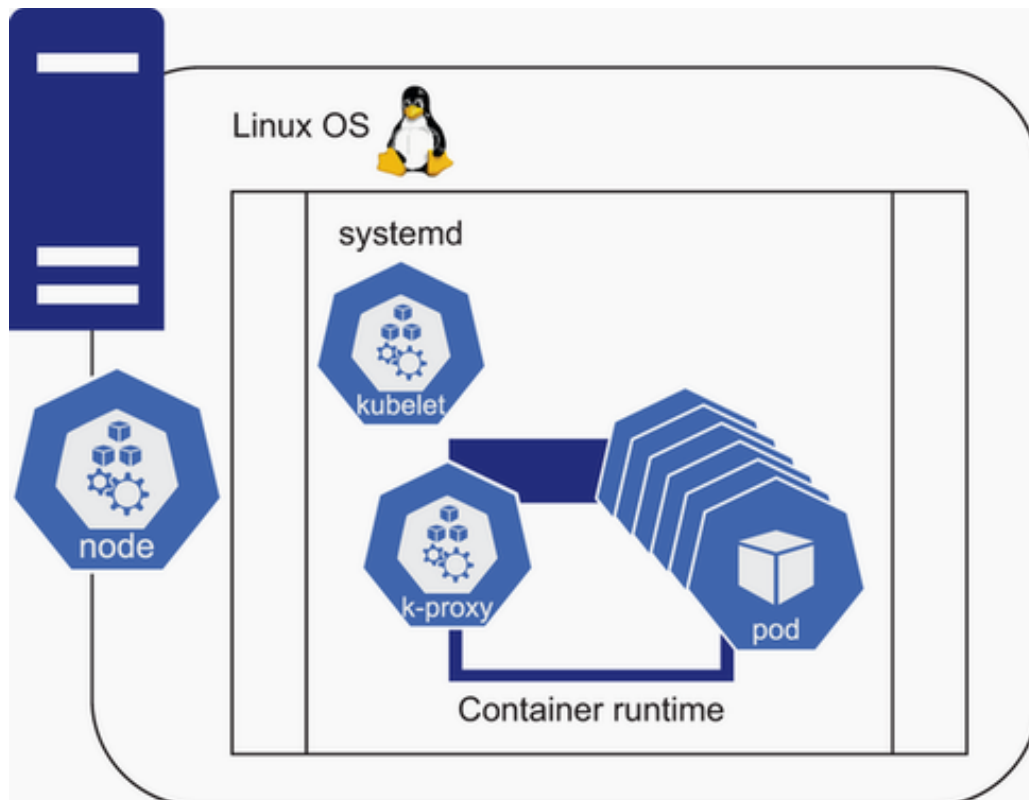


Figura 5.7: Componentes internos de un nodo de Kubernetes ejecutándose sobre Linux (Figura 2.4).

- Recolectar la basura (*garbage collection*) de los Pods cuando sea necesario, eliminando almacenamiento efímero o dispositivos de red huérfanos.

El kubelet no realiza el trabajo de aislamiento de forma directa; depende de una interfaz de tiempo de ejecución de contenedores (CRI) y de un proveedor de CNI para reconciliar el estado del nodo con los deseos del plano de control. Por ejemplo, si el plano de control decide desplegar un contenedor, el kubelet instruye al proveedor de CRI para que descargue la imagen del registro y asigne una dirección IP dentro del rango podCIDR del nodo.

Servicios y el Proxy de Red (kube-proxy)

El Service (Servicio) es otro objeto fundamental de la API. El componente binario kube-proxy se encarga de gestionar la creación y el mapeo de red de estos servicios en cada nodo. Los tipos principales de servicios son:

- ClusterIP: Un servicio interno que balancea la carga entre los Pods dentro del clúster.
- NodePort: Abre un puerto estático en la IP de cada nodo para exponer y balancear la carga hacia los Pods externamente.
- LoadBalancer: Crea un balanceador de carga externo al clúster (generalmente integrado con el proveedor de la nube).

El kube-proxy facilita que cada nodo del clúster conozca la ubicación de cada servicio y cada Pod, soportando protocolos de red TCP, UDP y SCTP, así como el reenvío de tráfico.

Nota sobre redes en Kubernetes

La conectividad de red se proporciona mediante una solución de software llamada proveedor de CNI. Actualmente, algunos proveedores avanzados de CNI están desarrollando arquitecturas de software que reemplazan por completo a kube-proxy, permitiendo gestionar el tráfico de los servicios de forma más eficiente sin necesidad de utilizar las reglas tradicionales de iptables de Linux.

5.2.7. El objeto Node de la API (The Node API object)

Los nodos proporcionan el soporte físico o virtual para los Pods, mientras que el plano de control define el grupo de nodos encargados de ejecutar los controladores del sistema, el administrador de controladores y el programador. Para listar los nodos de un clúster, se utiliza el siguiente comando básico:

```
$ kubectl get no                                     # 1. Forma abreviada de 'kubectl get nodes'
NAME          STATUS    ROLES    AGE   VERSION
kind-control-plane  NotReady  master   25s   v1.17.0 # 2. Salida de un clúster local 'kind'
```

Para inspeccionar detalladamente la estructura del objeto Node en la API, se puede solicitar su salida en formato YAML (`kubectl get no kind-control-plane -o yaml`). A continuación, se desglosa el manifiesto de la API dividido en sus secciones principales:

Metadatos y Especificaciones (metadata y spec)

```
apiVersion: v1
kind: Node
metadata:
  annotations:
    kubeadm.alpha.kubernetes.io/cri-socket: /run/containerd/containerd.sock # 1. Socket de C
    node.alpha.kubernetes.io/ttl: "0"
    volumes.kubernetes.io/controller-managed-attach-detach: "true"
  creationTimestamp: "2020-09-20T14:51:57Z"
  labels: # 2. Etiquetas e
    beta.kubernetes.io/arch: amd64
    beta.kubernetes.io/os: linux
    kubernetes.io/arch: amd64
    kubernetes.io/hostname: kind-control-plane
    kubernetes.io/os: linux
    node-role.kubernetes.io/master: ""
  name: kind-control-plane
spec:
  podCIDR: 10.244.0.0/24 # 3. Rango IP as
```

```
podCIDRs:
- 10.244.0.0/24
```

Estado y Condiciones del Nodo (status)

La sección status es actualizada continuamente por el kubelet hacia el *API server* para reportar la capacidad del host y sus condiciones de salud:

```
status:
  addresses:
    - address: 172.17.0.2
      type: InternalIP
    - address: kind-control-plane
      type: Hostname
  allocatable:          # Recursos disponibles para asignar a los Pods
    cpu: "2"
    memory: 2039264Ki
    pods: "110"
  capacity:            # Capacidad total real del hardware
    cpu: "2"
    memory: 2039264Ki
    pods: "110"
  conditions:          # Presiones del sistema detectadas por el kubelet
    - reason: KubeletHasSufficientMemory
      status: "False"    # False indica que el nodo está saludable y sin presión de memoria
      type: MemoryPressure
    - reason: KubeletHasNoDiskPressure
      status: "False"
      type: DiskPressure
    - reason: KubeletHasSufficientPID
      status: "False"
      type: PIDPressure
    - reason: KubeletReady
      status: "True"     # True indica que el nodo está listo para recibir cargas de trabajo
      type: Ready
```

Imágenes e Información del Sistema (images y nodeInfo)

El nodo almacena un registro de todas las imágenes de contenedores que se encuentran descargadas en su almacenamiento local, incluyendo los componentes centrales del propio Kubernetes:

```
images:
- names: ["k8s.gcr.io/etcd:3.4.3-0"]          # Base de datos del clúster
  sizeBytes: 289997247
- names: ["k8s.gcr.io/kube-apiserver:v1.17.0"] # Servidor de la API
```

```

    sizeBytes: 144347953
- names: ["k8s.gcr.io/kube-proxy:v1.17.0"]          # Proxy de red de servicios
  sizeBytes: 132100734
- names: ["docker.io/kindest/kindnetd:0.5.4"]      # Proveedor de red (CNI)
  sizeBytes: 113207016
nodeInfo:                                           # Versiones del entorno y del SO
  containerRuntimeVersion: containerd://1.3.2
  kernelVersion: 4.19.76-linuxkit
  kubeletVersion: v1.17.0
  operatingSystem: linux
  osImage: Ubuntu 19.10
--

```

5.2.8. Controladores y bucles de control (Controllers and control loops)

El término “control” está muy presente en Kubernetes. El sistema está compuesto por múltiples binarios ejecutables llamados controladores (como el kubelet, kube-proxy o el scheduler). Estos controladores se programan bajo un patrón de diseño de ingeniería de *software* conocido como bucles de control (*control loops*).

A grandes rasgos, Kubernetes actúa como una máquina de reconciliación de estados infinitamente activa, operando de manera similar a un termostato de aire acondicionado: mide constantemente el estado actual del sistema, lo compara con el estado deseado (definido por el usuario en los archivos YAML) y realiza las acciones necesarias para corregir cualquier desviación.

En lugar de regular la temperatura, los bucles de control de Kubernetes regulan aspectos críticos de los sistemas distribuidos:

- Vincular y montar de forma dinámica el almacenamiento persistente a los procesos.
- Crear contenedores en ejecución y escalar el número de réplicas bajo demanda.
- Terminar y migrar contenedores hacia nodos sanos si se detectan fallos de salud.
- Configurar rutas IP hacia puertos específicos.
- Actualizar dinámicamente los *endpoints* de los balanceadores de carga.

Reconciliación de los requisitos previos

Vinculando todo lo aprendido hasta ahora: los Pods nos proveen el instrumento para desplegar imágenes; el kubelet gestiona localmente su ciclo de vida dentro del nodo; los objetos Service y el balanceo interno son administrados por el kube-proxy; y un sistema DNS (como CoreDNS) permite el descubrimiento entre aplicaciones para que, por ejemplo, el microservicio de *Zeus Zap* localice y consuma la base de datos *CockroachDB*. Para el almacenamiento persistente, la combinación del namespace de Linux *mnt* junto con el kubelet permite montar dinámicamente un disco duro al Pod durante su creación.

Sin embargo, aún faltan resolver preguntas clave de arquitectura global: ¿Qué ocurre si un nodo entero muere? ¿Cómo se decide inicialmente a qué nodo específico debe asignarse un Pod? Para dar respuesta a estas necesidades de orquestación distribuida, entra en juego el plano de control (*control plane*).

5.2.9. Nuestra aplicación web y el plano de control (Our web application and the control plane)

Una vez establecidos los Pods y los nodos, el siguiente paso lógico es resolver los requisitos arquitectónicos complejos, tales como la alta disponibilidad (*HA - High Availability*). La alta disponibilidad no consiste simplemente en un mecanismo de conmutación por error (*failover*), sino en el cumplimiento estricto de un Acuerdo de Nivel de Servicio (*SLA - Service-Level Agreement*).

La disponibilidad del sistema se mide habitualmente mediante el “número de nueves” de tiempo de actividad (*uptime*):

- Cuatro nueves (99.99%): Permiten un máximo de 52 minutos y 36 segundos de inactividad al año.
- Cinco nueves (99.999%): Reducen el margen a tan solo 5 minutos y 15 segundos de inactividad anual (lo que equivale a apenas 26.25 segundos al mes).

Alcanzar un nivel de disponibilidad de cinco nueves significa que el sistema debe ser capaz de recuperarse de forma casi instantánea. Junto a la alta disponibilidad, coexisten otros objetivos operacionales críticos que Kubernetes ayuda a gestionar bajo la misma plataforma:

- a) Escalabilidad elástica e inmediata.
- b) Reducción y optimización de costos de infraestructura.
- c) Control de versiones de los contenedores para despliegues y regresiones (*rollbacks*).
- d) Seguridad integral, tanto para los usuarios como en la comunicación entre aplicaciones.

Nota sobre el diseño de aplicaciones

Si bien Kubernetes proporciona de forma nativa todas estas capacidades de resiliencia y escalado, las aplicaciones deben estar diseñadas para alinearse y soportar los patrones arquitectónicos nativos de la nube bajo los cuales opera la plataforma.

El punto de partida de todo este engranaje es el aprovisionamiento automatizado de los Pods. A partir de ahí, el plano de control asume la responsabilidad de actuar como el cerebro del clúster, coordinando la tolerancia a fallos y la distribución eficiente de las cargas de trabajo para maximizar el ahorro económico.

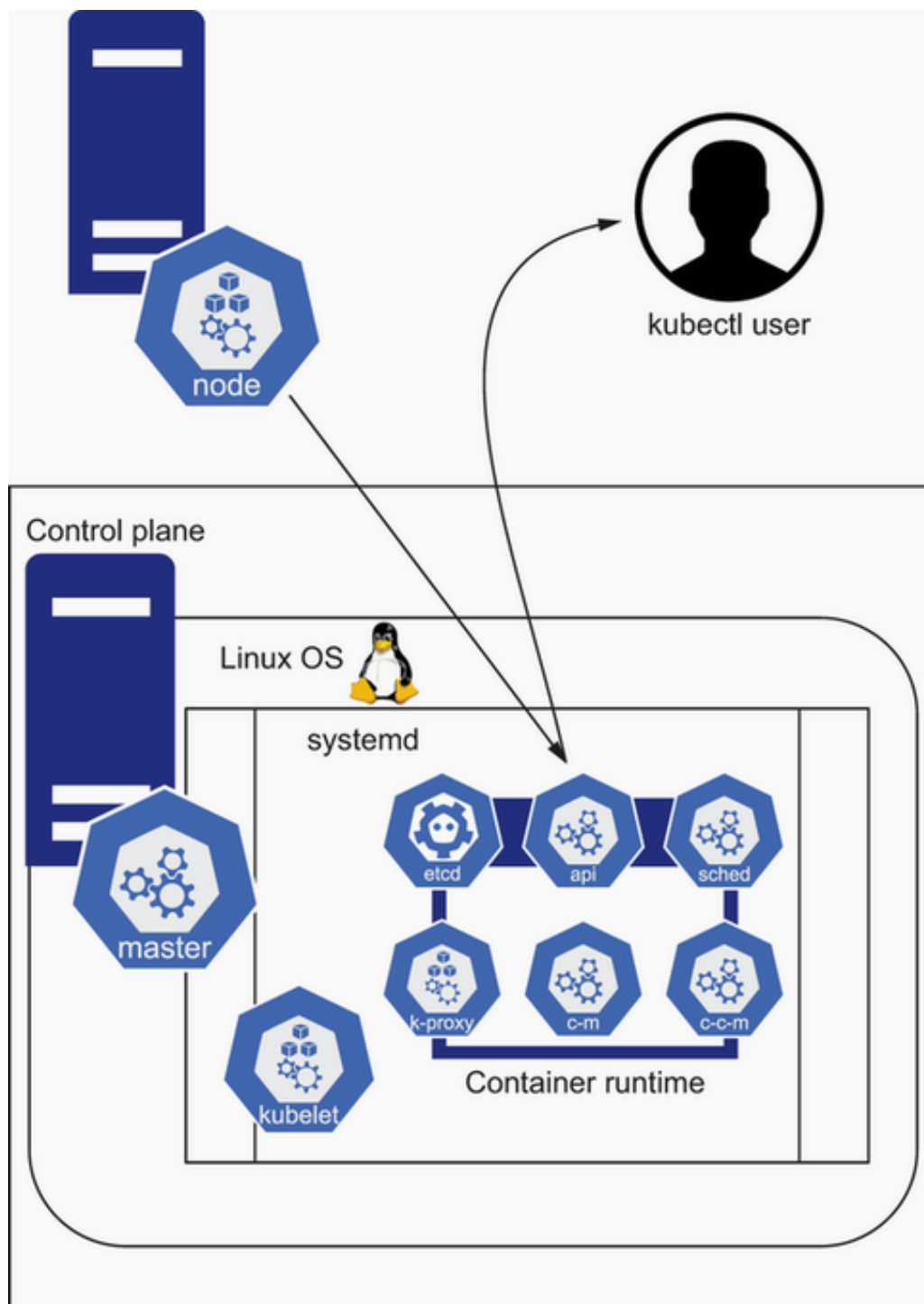


Figura 5.8: Componentes principales que integran el plano de control (Control Plane) de Kubernetes (Figura 2.5).

5.2.10. Creación de una aplicación web con kubectl (Creating a web application with kubectl)

Para comprender cómo el plano de control gestiona y facilita operaciones complejas -como el escalado automático y la tolerancia a fallos-, analizaremos lo que sucede internamente al ejecutar el comando básico: `kubectl apply -f deployment.yaml`.

El siguiente manifiesto representa la definición YAML estándar para este objeto de tipo *Deployment*:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3                                # Número deseado de copias concurrentes
  selector:
    matchLabels:
      app: nginx
  template:                                  # Plantilla para la creación de los Pods
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.7.9                 # Imagen OCI y versión específica
          ports:
            - containerPort: 80              # Puerto expuesto dentro del contenedor
```

En el momento exacto en que se ejecuta el comando `kubectl apply`, la herramienta de línea de comandos procesa el archivo y se comunica directamente con el primer componente y la puerta de entrada principal del plano de control: el ****servidor de la API**** (*API server*).

5.2.11. El servidor de la API de Kubernetes: kube-apiserver

El servidor de la API de Kubernetes, `kube-apiserver`, es un servidor REST basado en HTTP que expone todos los objetos de la API de un clúster, desde Pods y nodos hasta componentes como el *Horizontal Pod Autoscaler*. Este componente valida las peticiones y proporciona la interfaz web necesaria para realizar operaciones CRUD sobre el estado compartido del clúster.

En entornos de producción, el plano de control suele ofrecer una infraestructura robusta para el *API server*. Esto implica ejecutarlo de manera redundante en cada nodo del plano de control o delegarlo en un servicio de nube gestionado que garantice su alta disponibilidad. En la práctica, el acceso a la API se canaliza a través de un balanceador de carga de la nube o un proxy como *HAProxy*.

Todos los componentes del plano de control se comunican exclusivamente a través del *API server*:

- Cuando un nodo arranca, su agente kubelet se registra en el clúster comunicándose con la API.
- Todos los controladores del plano de control ejecutan un bucle de control con funcionalidad de vigilancia (*watch*) para monitorizar en tiempo real cualquier cambio en los objetos de la API.

Persistencia y Seguridad

El *API server* es el único componente del plano de control autorizado para comunicarse directamente con etcd, la base de datos de Kubernetes. Aunque en el pasado algunos componentes de red (CNI) accedían a ella de forma directa, la arquitectura actual centraliza todo a través de la API, proporcionando una interfaz con estado para cualquier modificación del clúster. Por este motivo, proteger los puntos de acceso HTTPS del *API server* es un pilar crítico en la seguridad del sistema.

Al operar en un plano de control de alta disponibilidad, todas las instancias del *API server* están activas y reciben tráfico simultáneamente, ya que se trata de un componente principalmente sin estado (*stateless*).

Para evitar accesos no autorizados, el servidor de la API integra controladores de admisión (*admission controllers*) encargados de validar la autenticación y la autorización de cada cliente. Es común integrar sistemas externos de identidad mediante el uso de *webhooks* (APIs HTTP PUSH para llamadas de retorno o *callbacks*).

Una vez que la llamada de *kubectl* es correctamente autenticada y autorizada, el *API server* persiste el nuevo objeto *Deployment* en etcd. El siguiente paso en el flujo del plano de control es activar el programador (*scheduler*) para asignarle un hogar físico a los nuevos Pods en algún nodo del clúster.

5.2.12. El programador de Kubernetes: kube-scheduler

La planificación distribuida de procesos es una tarea compleja. El programador de Kubernetes, kube-scheduler, proporciona una implementación limpia y directa para asignar recursos dentro del clúster, evaluando múltiples factores antes de decidir en qué nodo debe ejecutarse un Pod:

- Recursos de hardware: Capacidad disponible de CPU y memoria en cada nodo del clúster.
- Restricciones de políticas: Reglas de cumplimiento y cuotas establecidas para la programación.
- Afinidad y antiafinidad de Pods (*Pod affinity / anti-affinity*): Las reglas de afinidad atraen magnéticamente a los Pods hacia nodos específicos que cumplen ciertas características, mientras que las de antiafinidad los repelen para evitar, por ejemplo, que dos réplicas de la misma aplicación crítica compartan el mismo host físico.

- Tolerancias y rechazos (*Taints and tolerations*): Los *taints* permiten a un nodo repeler activamente un conjunto de Pods a menos que estos cuenten con una tolerancia específica, ayudando al programador a delimitar qué cargas de trabajo no deben mezclarse.

Retomando el ejemplo de la empresa *Zeus Zap*, el manifiesto define tres réplicas para el Pod. Estas copias aseguran tanto la tolerancia a fallos como la capacidad de escalar; es el kube-scheduler el encargado de determinar de forma óptima sobre qué nodo o nodos específicos se distribuirán dichas réplicas.

El ciclo de vida del Pod y aislamiento del plano de control

Una vez que el kube-scheduler toma la decisión y actualiza el manifiesto del Pod inyectándole el campo `nodeName`, el kubelet de ese nodo específico toma el control del ciclo de vida del Pod y procede a desplegarlo localmente, actuando como un “mini-programa del host.

El plano de control se encuentra completamente aislado y desacoplado de los nodos de trabajo (*worker nodes*) que ejecutan las aplicaciones. Esto tiene implicaciones críticas de resiliencia:

- Tolerancia a fallos del plano de control: Si ocurre una caída total del plano de control, la aplicación de *Zeus Zap* no perderá información ni se interrumpirá. El sitio web seguirá activo y procesando tráfico de forma normal en los nodos de trabajo.
- Limitaciones temporales: Durante la caída del plano de control, simplemente se congela el estado del clúster: no se pueden desplegar nuevas aplicaciones ni realizar cambios de configuración, pero la infraestructura existente se mantiene intacta.

En caso de que las aplicaciones requieran el uso de un disco persistente, el almacenamiento continuará operando de manera independiente. Si un nodo de trabajo llegase a fallar de forma simultánea a una interrupción del plano de control, en cuanto este último vuelva a estar en línea, los controladores automatizados de Kubernetes se encargarán de realizar una migración segura tanto de los datos persistentes como de la aplicación hacia un nuevo nodo sano.

5.2.13. Controladores de infraestructura y el patrón Operator (Infrastructure controllers)

Ciertas aplicaciones con estado (*stateful*), como la base de datos distribuida *CockroachDB*, presentan requisitos operacionales complejos. Para gestionarlas de forma nativa en Kubernetes, se utiliza el patrón **Operator** (Operador), el cual se ha convertido en el estándar para desplegar componentes avanzados, reemplazando el uso de manifiestos YAML planos.

El siguiente ejemplo ilustra la instalación del Operador para *CockroachDB*:

```
# 1. Instala las Definiciones de Recursos Personalizados (CRDs) del Operador
$ kubectl apply -f https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/in

# 2. Instala el binario del Operador en el espacio de nombres por defecto
$ kubectl apply -f https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/in
```

Definiciones de Recursos Personalizados (CRDs)

Las **Custom Resource Definitions (CRDs)** son objetos de la API que permiten definir e introducir nuevos objetos personalizados dentro del propio servidor de la API de Kubernetes. Una vez aplicado el CRD, el clúster es capaz de entender y gestionar un nuevo tipo de recurso.

Tras descargar el archivo de ejemplo del operador (`curl -LO ...`), se puede desplegar un clúster de base de datos utilizando este recurso personalizado:

```
apiVersion: crdb.cockroachlabs.com/v1alpha1
kind: CrdbCluster
metadata:
  name: cockroachdb
spec:
  dataStore:
    pvc:
      spec:
        accessModes: [- ReadWriteOnce]
        resources:
          requests: {storage: "60Gi"}
          volumeMode: Filesystem
  resources:
    requests: {cpu: "2", memory: "8Gi"}
    limits: {cpu: "2", memory: "8Gi"}
  tlsEnabled: true
  image:
    name: cockroachdb/cockroach:v21.1.5      # 1. Contenedor de la base de datos
  nodes: 3
  additionalLabels:
    crdb: is-cool
```

A partir de este bloque, el Operador se encarga de automatizar la creación de cientos de líneas de configuración subyacente: claves TLS almacenadas en *Secrets*, un *StatefulSet*, almacenamiento persistente (*PersistentVolumeClaims*), servicios de red y presupuestos de interrupción de Pods (*PDB*).

--

5.2.14. El Administrador de Controladores: KCM y la evolución hacia el CCM

Para dar vida a abstracciones como el almacenamiento persistente (`PersistentVolume` y `PersistentVolumeClaim`), Kubernetes se apoya en el **Kubernetes Controller Manager (KCM)** y el **Cloud Controller Manager (CCM)**.

El KCM es un binario único que ejecuta múltiples bucles de control de forma simultánea. Históricamente, el KCM albergaba todo el código necesario para interactuar con los diferentes proveedores de infraestructura (GKE, AWS, vSphere, etc.). Sin embargo, la inclusión de código específico de cada proveedor dentro del repositorio principal de Kubernetes (`kubernetes/kubernetes` o `kk`) volvió el código fuente difícil de mantener.

El nacimiento del Cloud Controller Manager (CCM)

Para desacoplar a Kubernetes de tecnologías propietarias y hacerlo completamente agnóstico de la nube, el equipo de desarrollo (bajo la CNCF) introdujo el **CCM**. El CCM extrae la lógica específica de los proveedores de nube hacia repositorios independientes mantenidos por terceros (como DigitalOcean, AWS, etc.), permitiendo que estos avancen a mayor velocidad sin modificar el núcleo de Kubernetes.

Cuando el clúster se ejecuta en una nube pública o privada, el CCM intercepta las peticiones y ejecuta las llamadas necesarias hacia las APIs del proveedor a través de cuatro controladores principales:

- **Node controller:** Sincroniza y valida el estado de los nodos con la infraestructura de la nube.
- **Route controller:** Configura las rutas de red y pasarelas en la infraestructura subyacente.
- **Service controller:** Gestiona la creación, actualización y eliminación de los balanceadores de carga de la nube.
- **Volume controller:** Orquesta la creación, el enganche (*attach*) y el montaje de volúmenes de almacenamiento en la nube.

Esta modularidad se complementa con otras interfaces estandarizadas y neutrales dentro de la arquitectura de Kubernetes:

- **CNI (Container Networking Interface):** Provee y asigna direcciones IP a los Pods.
- **CRI (Container Runtime Interface):** Permite conectar diferentes motores de ejecución de contenedores.
- **CSI (Container Storage Interface):** Permite a los proveedores de almacenamiento añadir soporte para nuevos tipos de discos sin alterar el código base del sistema.

Flujo de aprovisionamiento de recursos

Volviendo al caso de uso de *Zeus Zap*, cuando el Operador solicita el despliegue de las réplicas de la base de datos, ocurre lo siguiente:

- a) El kube-scheduler asigna el Pod a un nodo específico.
- b) El KCM (o CCM) detecta la solicitud de almacenamiento persistente, interactúa con la API de la infraestructura para aprovisionar el disco y lo monta en el servidor host.
- c) El agente kubelet levanta el Pod y, utilizando el namespace de Linux mnt, conecta de forma segura dicho almacenamiento dentro del contenedor.

Para exponer el sitio web público mediante un balanceador de carga externo, el usuario simplemente define un objeto de tipo Service configurado como type: LoadBalancer:

```
apiVersion: v1
kind: Service
metadata:
  name: example-service
spec:
  selector:
    app: example
  ports:
    - port: 8765
      targetPort: 9376
  type: LoadBalancer                                # Solicita un balanceador externo
```

El bucle de vigilancia (*watch loop*) del CCM detecta esta configuración, aprovisiona una dirección IP pública mediante las APIs de la nube y la vincula a los nodos del clúster. Finalmente, cuando el tráfico de los usuarios llega a los nodos del hardware externo, la red interna administrada por el proveedor de **CNI** se encarga de enrutar el paquete de forma transparente hasta el Pod correcto.

5.2.15. Escalado, alta disponibilidad y gestión de fallos (Scaling, HA, and the control plane)

El escalado elástico de las aplicaciones es el mecanismo subyacente que permite la alta disponibilidad (*HA*) en la nube y, de forma más optimizada, en Kubernetes. Ante picos de demanda o fallos en la infraestructura, el sistema necesita instanciar más Pods o redesplosarlos en nodos sanos para mantener el nivel de servicio.

El comando `kubectl scale` permite incrementar o disminuir de forma inmediata el número de Pods en ejecución, actuando directamente sobre controladores como *ReplicaSets* o *StatefulSets*:

```
$ kubectl scale --replicas 300 deployment zeus-front-end-ui
```

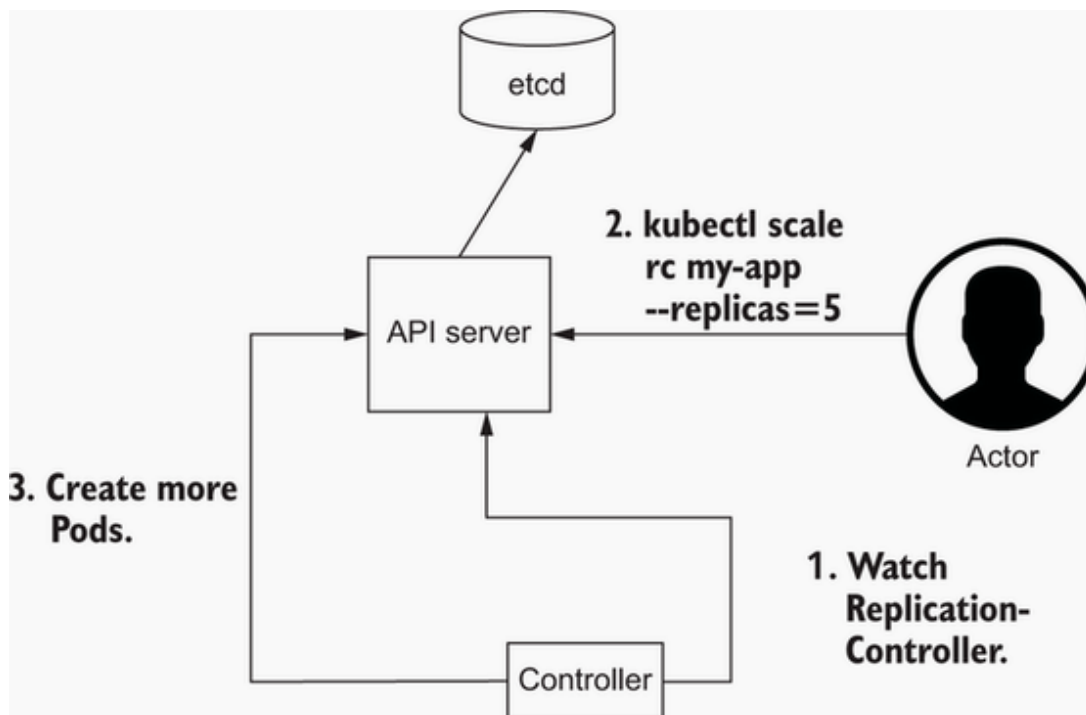


Figura 5.9: Secuencia típica y flujo de comunicación en el plano de control al ejecutar el comando `kubectl scale` (Figura 2.6).

Nota sobre los DaemonSets

Este comando de escalado no aplica a los objetos de tipo `DaemonSet`. Por definición, un `DaemonSet` ejecuta exactamente un Pod en cada nodo del clúster; por lo tanto, su escala está determinada estrictamente por el número de nodos físicos o virtuales que componen la infraestructura.

En el escenario de Zeus Zap, el comando anterior ajusta la cantidad de Pods que respaldan el *frontend* de la UI, siguiendo el mismo flujo de comunicación entre el *scheduler*, el KCM y el kubelet.

Niveles de tolerancia a fallos y resiliencia

Los fallos en producción son inevitables. Kubernetes clasifica y gestiona las acciones de recuperación ante incidentes en tres niveles operativos distintos:

- a) Cortes a nivel de Pod: El kubelet es el guardián del ciclo de vida del Pod en el host. Si el proceso del contenedor se detiene o si falla una prueba de vida (*liveness probe*) definida por el usuario, el kubelet inicia un bucle de control para reiniciar automáticamente el Pod dañado.
- b) Cortes a nivel de Nodo: El kubelet envía constantemente una señal de latido (*heartbeat*) al *API server* a través del espacio de nombres `kube-node-lease`. Si un nodo deja de enviar este latido debido a un fallo de red o de hardware, el KCM cambia su estado a fuera de línea (*offline*). Inmediatamente, se detiene la programación de nuevas cargas en él, y los Pods que residían allí se marcan para eliminación y se reprograman de forma automática en nodos sanos. Este comportamiento se puede

emular manualmente utilizando los comandos `kubectl cordon` (marcar como no programable) y `kubectl drain` (evacuar el nodo).

- c) Actualizaciones de software sin interrupciones: Kubernetes está diseñado para realizar actualizaciones de la plataforma o de las versiones de las aplicaciones sin tiempo de inactividad (*zero-downtime*).

Condición crítica: Durabilidad de la aplicación

*Para evitar la pérdida de datos durante los despliegues o fallos, el software que se ejecuta sobre Kubernetes debe ser resiliente y soportar un apagado e inicio gradual (*graceful shutdown and startup*). Si una instancia se destruye a mitad de una transacción, la lógica de la aplicación debe ser capaz de reintentar dicha transacción en otra réplica o reanudarla de forma segura al reiniciarse.*

Actualizar la versión de un *Deployment* es tan sencillo como modificar la etiqueta de la imagen en el archivo YAML. En el día a día, esto se realiza mediante tres comandos principales de la API:

- `kubectl edit`: Abre el manifiesto del objeto directamente en un editor de texto de la terminal local para realizar cambios en caliente.
- `kubectl apply`: Toma un archivo de configuración local, contrasta las diferencias y reemplaza automáticamente el objeto modificado en el clúster.
- `kubectl patch`: Permite aplicar un pequeño fragmento de código (parche) para actualizar atributos muy específicos de un objeto sin necesidad de reescribir todo el archivo.

5.2.16. Escalado automático (Autoscaling)

Escalar los despliegues de forma manual es una excelente opción para gestionar cambios previstos, pero resulta inviable ante un incremento repentino de tráfico, como recibir 10,000 peticiones web nuevas por minuto. Para resolver este escenario, Kubernetes ofrece mecanismos de escalado automático (*autoscaling*) que pueden configurarse en tres dimensiones distintas:

- Incrementar el número de Pods (Escalado Horizontal): Gestionado por el objeto de la API `HorizontalPodAutoscaler` (HPA), el cual duplica o multiplica las instancias de los Pods según la demanda de tráfico o uso de CPU.
- Asignar más recursos a los Pods existentes (Escalado Vertical): Gestionado por el `VerticalPodAutoscaler` (VPA), el cual incrementa dinámicamente los límites de CPU y memoria de los Pods en ejecución.
- Añadir más nodos al clúster (Escalado de Infraestructura): Gestionado por el `Cluster Autoscaler`, el cual aprovisiona automáticamente nuevos servidores (nodos) cuando los recursos globales del clúster se agotan y los Pods no encuentran espacio para ejecutarse.

Nota sobre plataformas locales (*Bare Metal*)

Dependiendo de la arquitectura de la infraestructura, los componentes de escalado automático (especialmente el Cluster Autoscaler) podrían no estar disponibles de forma nativa en entornos que utilicen servidores físicos propios (bare metal), requiriendo integraciones a medida con las APIs de virtualización locales.

5.2.17. Gestión de costos y densidad de Pods (Cost management)

Cuando un clúster escala automáticamente (*autoscaling*), añade nuevos nodos a la infraestructura, lo que incrementa de forma directa los costos de facturación en la nube. Aunque disponer de más nodos permite que las aplicaciones tengan más réplicas y soporten una mayor carga de trabajo, la optimización financiera exige buscar soluciones de ahorro. Aquí es donde entra en juego el concepto de ****densidad de Pods**** (*Pod density*), que consiste en empaquetar de forma densa y eficiente los Pods dentro de los nodos existentes.

Dado que los Pods son la unidad básica de cómputo y los nodos (físicos o virtuales) representan el costo real del servidor, cuantos más Pods compartan un mismo nodo, menor será el gasto en servidores adicionales. Kubernetes permite alcanzar una alta densidad mediante el sobreaprovisionamiento de los hosts controlando los siguientes pasos esenciales:

- a) Dimensionar y perfilar las aplicaciones: Es imprescindible probar y perfilar cada aplicación para conocer su consumo real de CPU y memoria. Una vez analizadas, se deben configurar adecuadamente las restricciones y límites de recursos (*resource limits*) en sus manifiestos de Kubernetes.
- b) Seleccionar el tamaño óptimo de los nodos: Elegir tamaños de máquinas virtuales o servidores *bare metal* con capacidades de hardware heterogéneas permite consolidar múltiples aplicaciones en un mismo host. A pesar de buscar la máxima densidad, se debe mantener un número mínimo de nodos que garantice la alta disponibilidad exigida por los acuerdos de nivel de servicio (*SLA*).
- c) Agrupar aplicaciones estratégicamente: Consiste en rellenar los huecos de recursos combinando cargas de trabajo grandes con aplicaciones más pequeñas (análogo a llenar un frasco con canicas y luego verter arena en los espacios vacíos). Las restricciones de asignación como los *taints* (rechazos) y *tolerations* (tolerancias) permiten a los operadores controlar este empaquetado.

Mitigación del vecino ruidoso (*Noisy Neighbors*) y apagado programado

El empaquetado denso introduce el riesgo del fenómeno del vecino ruidoso, donde aplicaciones con picos agresivos de consumo compiten por recursos escasos afectando a los contenedores adyacentes. Para mitigar esto, Kubernetes permite distribuir estas aplicaciones problemáticas de manera uniforme a lo largo del clúster utilizando reglas de ****afinidad y antiafinidad de Pods**** (*Pod affinity / anti-affinity*).

Adicionalmente, se pueden lograr mayores ahorros económicos combinando el escalado automático con el uso de máquinas virtuales efímeras en la nube (*Spot/Ephemeral VMs*), o implementando políticas de apagado programado. Si los clústeres dedicados a entornos de Desarrollo o Control de Calidad (*QA*) no se utilizan durante los fines de semana, la mejor estrategia de optimización consiste simplemente en reducir a cero el conteo de los nodos de trabajo (*worker nodes*) y volver a escalarlos al inicio de la jornada laboral.

5.2.18. Resumen del capítulo (Summary)

- El Pod como unidad fundamental: El Pod es el objeto básico de la API de Kubernetes que utiliza los *namespaces* de Linux para crear un entorno aislado en el que se ejecutan uno o más contenedores de forma conjunta.
- Patrones de diseño de la API: Kubernetes fue diseñado para gestionar los Pods a través de diferentes patrones de abstracción de nivel superior, los cuales se representan como objetos de la API (tales como *Deployments*, *StatefulSets*, *DaemonSets* y *Jobs*).
- Controladores y gestión del ciclo de vida: Los controladores son los componentes de *software* encargados de crear, vigilar y mantener el ciclo de vida de los Pods. Entre los más críticos se encuentran el kubelet (agente del nodo), el *Cloud Controller Manager* (CCM) y el kube-scheduler (programador).
- El plano de control como el cerebro del sistema: A través de sus bucles de control distribuidos, el plano de control centraliza y automatiza tareas complejas de los sistemas distribuidos: vincula almacenamiento persistente a los procesos, escala el número de contenedores, destruye y migra instancias cuando están insalubres, configura rutas IP hacia los puertos y actualiza dinámicamente los *endpoints* de los balanceadores de carga.
- El Servidor de la API como punto de entrada único: El componente kube-apiserver actúa como un servidor REST que valida y proporciona la interfaz web para realizar operaciones CRUD sobre el estado compartido del clúster. En entornos de producción, se ejecuta de forma redundante en todos los nodos del plano de control para garantizar una arquitectura de alta disponibilidad (*HA*).